

République Tunisienne
Ministère de l'Enseignement Supérieur et de la Recherche Scientifique

Université Tunis El Manar
Institut Supérieur d'Informatique
ISI

MÉMOIRE DE MASTÈRE

Présenté dans le cadre du
Master Développement Logiciel

SmartRecruit

Conception et réalisation d'une plateforme de mise en
relation étudiants-entreprises assistée par un moteur de
recommandation intelligent

Réalisé par

Rahma Amiri

Code du stage	MDL/26/12
Organisme d'accueil	Best Solutions
Encadrant professionnel	Hammami Houssein
Période du stage	Du 02 mars 2026 au 02 juillet 2026

Année universitaire 2025–2026

Remerciements

Je remercie Best Solutions pour l'accueil de ce stage et pour le cadre professionnel dans lequel s'inscrit le projet SmartRecruit. Ce travail représente une occasion de relier les connaissances acquises en développement logiciel à une problématique concrète de mise en relation entre étudiants et entreprises.

J'adresse mes remerciements à mon encadrant professionnel, Hammami Houssein, pour l'encadrement du projet. La conception d'une plateforme de recrutement demande de mettre en relation plusieurs dimensions : les parcours des utilisateurs, l'organisation des données, l'intégration des services et l'interprétation des résultats produits par un algorithme.

Je remercie également l'Institut Supérieur d'Informatique de l'Université Tunis El Manar et l'équipe pédagogique du Master Développement Logiciel pour la formation qui constitue le socle de ce mémoire. Mes remerciements s'adressent enfin à mes proches pour leur soutien, ainsi qu'aux personnes qui consacreront du temps à la lecture et à l'évaluation de ce travail.

Résumé

La mise en relation entre étudiants et entreprises exige de rapprocher des profils hétérogènes et des offres dont les compétences ne sont pas toujours formulées avec le même vocabulaire. Une recherche littérale ignore des équivalences utiles, tandis qu'un score insuffisamment expliqué peut rendre la présélection difficile à discuter. Ce mémoire présente SmartRecruit, une plateforme conçue pour centraliser les profils, les curriculum vitæ, les offres et les candidatures, puis assister leur comparaison par un classement explicable.

Le prototype associe une application Laravel, une interface Blade, une base MySQL et un service Python consacré à l'extraction de texte et au matching. Le moteur combine une pondération TF-IDF, une similarité cosinus, la couverture de compétences et une taxonomie de domaines techniques. Un moteur PHP assure une continuité de calcul lorsque le service Python est indisponible. Le mémoire décrit les besoins, la conception des données et des échanges, l'implémentation des parcours et les mécanismes de recommandation.

L'évaluation repose sur l'examen du code et sur des sondes locales reproductibles. Elle confirme la présence du socle applicatif tout en révélant des écarts entre les deux moteurs, des limites de parsing et des points à corriger dans le stockage de secours et le déploiement. Les scores sont présentés comme des indicateurs de rapprochement, sans prétention à mesurer une probabilité de réussite professionnelle. Le travail propose enfin une trajectoire d'amélioration fondée sur la cohérence des contrats, la protection des données, la validation humaine et une évaluation de classement sur un corpus annoté.

Mots-clés : recrutement étudiant, recommandation, traitement automatique du langage, TF-IDF, similarité cosinus, Laravel, Python, explicabilité.

Abstract

Matching students with companies requires comparing heterogeneous profiles and job descriptions that often express related skills using different terms. Literal keyword searches may miss useful equivalences, while an unexplained score can make candidate screening difficult to assess. This dissertation presents SmartRecruit, a platform that centralizes profiles, résumés, opportunities and applications, and supports comparison through an interpretable ranking mechanism.

The prototype combines a Laravel application, a Blade interface, a MySQL database and a Python service for text extraction and matching. Its scoring mechanism uses TF-IDF weighting, cosine similarity, skill coverage and a taxonomy of technical domains. A PHP implementation provides a fallback when the Python service is unavailable. The dissertation covers requirements, data and interaction design, implementation and the recommendation pipeline.

Evaluation is based on source-code inspection and reproducible local probes. It confirms the application's foundations while identifying inconsistencies between the two engines, parsing limitations, and issues in fallback persistence and deployment. Scores are treated as matching indicators rather than estimates of professional success. Proposed improvements focus on consistent service contracts, data protection, human oversight and a ranking evaluation protocol using an independently annotated corpus. No large-scale performance benchmark or user-study result is claimed.

Keywords: student recruitment, recommendation, natural language processing, TF-IDF, cosine similarity, Laravel, Python, explainability.

Note de périmètre et de traçabilité

Le présent mémoire concerne le stage MDL/26/12, réalisé par Rahma Amiri au sein de Best Solutions du 02/03/2026 au 02/07/2026. Les informations institutionnelles et administratives de la couverture correspondent aux éléments communiqués pour ce travail. Aucune composition de jury, identité d'encadrant académique ou caractéristique commerciale de l'organisme d'accueil n'est supposée.

La description technique porte sur les fichiers du projet SmartRecruit disponibles lors de la rédaction. L'analyse initiale a été conduite le 10 septembre 2026 et les sondes accompagnant cette version du mémoire ont été rejouées le 11 septembre 2026. Ces vérifications postérieures au stage constituent une évaluation rétrospective du prototype ; elles ne sont pas présentées comme un journal d'exécution établi pendant le stage.

Trois niveaux de discours sont distingués : les fonctionnalités et règles observées dans le code, les comportements effectivement reproduits par des essais locaux, et les améliorations proposées. Une fonctionnalité visible dans le code n'est pas automatiquement qualifiée de validée de bout en bout. Les parcours avec une base MySQL réelle, les essais de charge et les études utilisateurs nécessitent un protocole complémentaire. Les scénarios synthétiques des annexes sont des exemples de test, sans données personnelles réelles.

Le dossier livré contient les sources LaTeX, la bibliographie et les scripts de sondes. Les figures sont des schémas techniques construits à partir de l'implémentation ou de la conception explicitement proposée. Elles ne sont pas présentées comme des captures d'un déploiement en production. Les écarts constatés sont intégrés à l'analyse critique afin de rendre le bilan vérifiable et utile à la poursuite du projet.

Contents

Remerciements	i
Résumé	ii
Abstract	iii
Note de périmètre et de traçabilité	iv
Liste des figures	ix
Liste des tableaux	x
Liste des abréviations	xi
Introduction générale	1
1 Contexte du projet et fondements scientifiques	3
1.1 Cadre académique et professionnel	3
1.2 Enjeux d’une plateforme de recrutement étudiant	4
1.2.1 Des informations hétérogènes à rendre comparables	4
1.2.2 Trois points de vue sur le même processus	4
1.3 Problématique et objectifs	5
1.3.1 Question centrale	5
1.3.2 Objectifs fonctionnels et techniques	6
1.4 État de l’art et approches de rapprochement	6
1.4.1 Gestion des candidatures et recommandation	6
1.4.2 Représentation lexicale par TF-IDF et cosinus	7
1.4.3 Word2Vec : apprendre des représentations de mots	7
1.4.4 BERT et contextualisation du langage	8
1.4.5 CamemBERT et prise en compte du français	8
1.4.6 Sentence-BERT et similarité entre textes	9

1.5	Positionnement retenu pour le prototype	9
1.5.1	Une combinaison explicable de signaux	9
1.5.2	Explicabilité, responsabilité et limites des preuves	10
2	Analyse et spécification des besoins	12
2.1	Problématique et périmètre fonctionnel	12
2.2	Acteurs et responsabilités	13
2.3	Spécification des cas d'utilisation	13
2.3.1	UC01 : créer un compte et ouvrir une session	15
2.3.2	UC02 : compléter le profil et déposer un CV	15
2.3.3	UC03 : publier et maintenir une offre	16
2.3.4	UC04 : candidater à une offre	16
2.3.5	UC05 : examiner un classement explicable	17
2.3.6	UC06 : suivre et qualifier une candidature	17
2.4	Exigences identifiées et critères d'acceptation	18
2.5	Contraintes, priorités et démarche rétrospective	20
3	Conception et architecture de la solution	21
3.1	Architecture générale observée	21
3.2	Organisation du noyau Laravel	22
3.3	Modèle relationnel et cardinalités	23
3.3.1	Entités persistées	23
3.3.2	Relations et garanties effectives	24
3.3.3	Champs structurés, JSON et évolution du schéma	26
3.4	Contrats d'API et flux de traitement	26
3.4.1	Interface entre Laravel et Python	27
3.4.2	Enchaînement du dépôt et du classement	27
3.5	Transactions, concurrence et cohérence	29
3.6	Architecture cible proposée	30
4	Réalisation et intégration de SmartRecruit	32
4.1	Du besoin fonctionnel au prototype exécutable	32
4.2	Organisation du code et responsabilités	32
4.2.1	Une application Laravel à organisation compacte	32

4.2.2	Circulation d'une requête	34
4.3	Authentification et maîtrise des accès	34
4.3.1	Connexion et création de compte actuelles	34
4.3.2	Rôles et contrôle de propriété	35
4.4	Parcours d'interface	35
4.4.1	Navigation et tableaux de bord	35
4.4.2	Profil étudiant et dépôt du CV	36
4.4.3	Offres et candidatures	36
4.5	Persistance et cohérence métier	37
4.6	Extraction des CV et intégration du moteur	38
4.6.1	Chaîne d'extraction	38
4.6.2	Calcul local et appel distant	38
4.7	Configurations d'exécution et déploiement	39
4.7.1	Exécution locale, Apache et conteneurs	40
4.7.2	Frontière entre démonstration et production	40
5	Conception et fonctionnement du moteur de matching	42
5.1	Organisation de la chaîne de traitement	42
5.2	Extraction et préparation du CV	42
5.2.1	Sources textuelles et extraction PDF	42
5.2.2	Compétences, formation et expérience	44
5.3	Normalisation, alias et catégories	45
5.3.1	Traitement lexical effectivement implémenté	45
5.3.2	Rôle du dictionnaire et de la taxonomie	46
5.4	Définition des composantes mathématiques	46
5.4.1	Couverture des compétences et des catégories	46
5.4.2	TF-IDF calculé sur une paire de documents	47
5.5	Formules finales PHP et Python	48
5.6	Exemple numérique reproductible	49
5.7	Contrat HTTP et mécanisme de repli	50
5.8	Divergences constatées et trajectoire de correction	51
6	Validation du prototype et perspectives d'amélioration	52

6.1	Périmètre et statut des résultats	52
6.2	Méthodologie de vérification	52
6.2.1	Trois niveaux de preuve	52
6.2.2	Reproductibilité et protection des données	53
6.3	Résultats des contrôles techniques locaux	53
6.4	Comportements du moteur de rapprochement	54
6.4.1	Sondes de cohérence entre PHP et Python	54
6.4.2	Expérience et qualité de l'extraction	55
6.5	Accès, sessions et frontière de déploiement	56
6.5.1	Exposition observée avec le serveur intégré	56
6.5.2	Confiance proxy et authentification	56
6.6	Cohérence métier et persistance : constats statiques	57
6.7	Protocole proposé d'évaluation de la pertinence	58
6.7.1	Constitution du corpus et annotations	58
6.7.2	Mesures de classement	59
6.7.3	Comparaisons et analyse des erreurs	59
6.8	Plan de validation complémentaire et progression	60
	Conclusion générale	61
	Bibliographie	63

List of Figures

2.1	Vue synthétique des cas d'utilisation de SmartRecruit	14
3.1	Architecture applicative observée et échanges entre composants	22
3.2	Modèle de données des sept tables de SmartRecruit	25
3.3	Séquence observée de consultation du classement d'une offre	28
4.1	Configuration de démonstration décrite par Docker Compose	39
5.1	Chaîne de préparation des données et de calcul du matching dans SmartRecruit	43

List of Tables

1.1	Positionnement des familles de méthodes pour SmartRecruit	10
2.1	Responsabilités principales des acteurs	14
2.2	Exigences fonctionnelles et observations vérifiables	18
2.3	Exigences de qualité à vérifier lors de la consolidation	19
3.1	Tables du modèle relationnel existant	23
4.1	Répartition des responsabilités dans le code disponible	33
5.1	Contributions en points pour l'exemple Java, React et SQL	49
6.1	Contrôles exécutés sur la version disponible en septembre 2026	54
6.2	Sondes ciblées de comparaison des deux implémentations	55
6.3	Progression proposée à partir de l'évaluation du prototype	60

Liste des abréviations

Sigle	Signification
API	Application Programming Interface : interface de programmation.
ATS	Applicant Tracking System : système de suivi des candidatures.
BERT	Bidirectional Encoder Representations from Transformers.
CI/CD	Intégration continue et livraison ou déploiement continu.
CRUD	Création, lecture, modification et suppression.
CSRF	Cross-Site Request Forgery : falsification de requête intersite.
CV	Curriculum vitæ.
HTTP	Hypertext Transfer Protocol.
IDF	Inverse Document Frequency : fréquence documentaire inverse.
JSON	JavaScript Object Notation : format d'échange de données.
MVP	Minimum Viable Product : version minimale viable.
NDCG	Normalized Discounted Cumulative Gain : gain cumulé actualisé normalisé.
NLP / TAL	Natural Language Processing / traitement automatique du langage.
OCR	Optical Character Recognition : reconnaissance optique de caractères.
ORM	Object-Relational Mapping : correspondance objet-relationnel.
PDF	Portable Document Format.
REST	Representational State Transfer : style d'architecture d'API.
SQL	Structured Query Language.
TF-IDF	Term Frequency–Inverse Document Frequency.
UML	Unified Modeling Language : langage de modélisation unifié.
WSGI	Web Server Gateway Interface : interface serveur Python.

Sigle	Signification
XSS	Cross-Site Scripting : injection de script dans une page web.

Introduction générale

L'accès à un stage ou à une première expérience professionnelle met en relation deux besoins complémentaires. Les étudiants cherchent des missions compatibles avec leur formation, leurs projets et les compétences qu'ils souhaitent approfondir. Les entreprises souhaitent identifier des profils capables de contribuer à leurs activités, tout en tenant compte du caractère formateur du stage. La rencontre entre ces besoins dépend de la qualité des informations disponibles et de la manière dont elles sont comparées.

Les curriculum vitæ et les descriptions d'offres constituent des documents riches, mais peu homogènes. Une technologie peut être citée par son nom, par un framework associé ou par une expression propre à une langue. Les expériences sont décrites selon des formats différents et les compétences déclarées n'ont pas toutes le même niveau de preuve. Une plateforme qui ne fait que stocker ces documents laisse à l'utilisateur l'intégralité du travail de rapprochement. À l'inverse, une plateforme qui réduit ce travail à une note opaque peut donner une impression de précision sans fournir les éléments permettant d'apprécier le résultat.

Ce mémoire étudie la problématique suivante : *comment concevoir une plateforme de mise en relation étudiants-entreprises qui automatise une partie de la comparaison entre profils et offres, tout en conservant des résultats explicables, une architecture évolutive et des limites identifiées ?* La réponse proposée prend la forme de SmartRecruit, développé dans le cadre du stage MDL/26/12 réalisé chez Best Solutions. Le projet associe la gestion des parcours de recrutement à un moteur de recommandation fondé sur des représentations textuelles et des connaissances explicites sur les compétences.

Le premier objectif est fonctionnel : regrouper les profils étudiants, les offres publiées, les candidatures et les décisions dans une application cohérente. Le deuxième objectif est technique : séparer les responsabilités de l'application web, de la persistance et du traitement de texte, de manière à pouvoir faire évoluer le moteur sans réécrire tous les parcours. Le troisième objectif est méthodologique : fournir une explication du score et une démarche d'évaluation qui permette de distinguer le comportement du logiciel de la pertinence réelle de ses recommandations.

Le choix initial s'oriente vers une méthode légère combinant TF-IDF, similarité cosinus et couverture de compétences. Cette méthode constitue une base de comparaison contrôlable : elle permet d'inspecter les termes, les synonymes et les pondérations utilisés. Elle ne résout cependant pas toutes les difficultés linguistiques. La négation, le

niveau de maîtrise, les compétences seulement évoquées et les documents numérisés comme images restent des sources d'erreur. L'architecture prévoit donc une possibilité d'évolution, sans confondre cette possibilité avec une intégration déjà réalisée.

L'apport du travail ne se limite pas au calcul d'une note. Il concerne aussi l'articulation entre cette note et les processus applicatifs : à quel moment la calculer, quelles données lui transmettre, comment la conserver, et comment réagir lorsque le service chargé du calcul devient indisponible. L'évaluation montre notamment qu'un mécanisme de secours n'est utile que si son comportement est suffisamment cohérent avec le service principal. De même, la continuité d'affichage ne garantit pas la continuité des données lorsque plusieurs stockages indépendants sont utilisés.

Le mémoire est organisé en six chapitres. Le chapitre 1 présente le contexte, la problématique et l'état de l'art. Le chapitre 2 formalise les besoins et les critères d'acceptation. Le chapitre 3 expose l'architecture, le modèle de données et les interactions. Le chapitre 4 décrit les éléments effectivement présents dans l'implémentation. Le chapitre 5 détaille les traitements et les formules du moteur. Enfin, le chapitre 6 confronte les choix aux vérifications disponibles, discute les limites et propose une progression vers une version plus fiable. Les annexes réunissent les contrats d'échange, des scénarios synthétiques et les éléments de reproductibilité.

CHAPTER 1

Contexte du projet et fondements scientifiques

1.1 Cadre académique et professionnel

Ce mémoire présente SmartRecruit, une plateforme de mise en relation entre étudiants et entreprises intégrant un moteur de rapprochement entre offres et profils. Il s'inscrit dans le cadre du Master en Développement Logiciel de l'Institut Supérieur d'Informatique (ISI), Université de Tunis El Manar. Le travail est porté par Rahma Amiri, dans le cadre d'un stage réalisé auprès de Best Solutions du 2 mars au 2 juillet 2026, sous l'encadrement professionnel de Hammami Housseem.

Le projet associe deux dimensions complémentaires du développement logiciel. La première concerne la construction d'une application métier : représenter les utilisateurs, gérer des offres, enregistrer des candidatures et présenter des informations cohérentes selon le rôle connecté. La seconde concerne le traitement de documents : extraire le texte d'un curriculum vitae, reconnaître des compétences, construire une représentation comparable et produire un score accompagné d'éléments d'explication. La valeur attendue provient de l'intégration de ces dimensions dans un parcours utilisable.

Best Solutions constitue l'organisme d'accueil du stage et le cadre de l'encadrement professionnel indiqué ci-dessus. La présentation du projet ne suppose aucune information supplémentaire sur l'effectif, l'historique, les clients ou l'organisation interne de cette structure. L'analyse porte sur le système SmartRecruit et sur ses choix de conception. Elle n'attribue pas à l'entreprise des pratiques de recrutement, des difficultés opérationnelles mesurées ou des résultats économiques qui ne sont pas documentés.

Le dépôt examiné fournit les éléments techniques mobilisés dans ce mémoire : code applicatif, configuration, microservice Python et documentation du projet. Ces artefacts permettent d'expliquer le fonctionnement observable du prototype. Ils ne constituent pas, à eux seuls, un journal détaillé des activités du stage. La rédaction distingue donc les fonctions présentes, les limites constatées et les évolutions proposées, sans reconstruire artificiellement une chronologie de réalisation.

1.2 Enjeux d'une plateforme de recrutement étudiant

1.2.1 Des informations hétérogènes à rendre comparables

Une offre exprime un besoin au moyen d'un intitulé, d'une description et de compétences recherchées. Un profil étudiant présente une formation, des projets, des expériences éventuelles et un ensemble de savoir-faire. Ces deux descriptions répondent à des intentions différentes : l'entreprise formule ses attentes tandis que le candidat décrit son parcours. Leur comparaison exige de construire des correspondances sans effacer cette différence de perspective.

Le vocabulaire technique constitue une première difficulté. Une offre peut demander Java, alors qu'un candidat mentionne Spring Boot ; une autre peut employer une formulation française pour une compétence généralement nommée en anglais. Une recherche strictement littérale distingue ces expressions, même lorsqu'un rapprochement peut être pertinent. Inversement, assimiler toutes les technologies voisines serait excessif : l'utilisation d'un framework ne démontre pas la maîtrise de toutes les composantes de son écosystème. Le système doit donc rendre explicite la granularité de ses rapprochements.

La structure des documents introduit une seconde difficulté. Un CV textuel, un PDF contenant du texte et un document numérisé sous forme d'image ne fournissent pas les mêmes possibilités d'extraction. Une mise en page à plusieurs colonnes peut aussi perturber l'ordre de lecture. Le score ne peut être plus fiable que les informations effectivement accessibles au moteur. Une absence de compétence détectée peut correspondre à une absence réelle dans le document, à une formulation inconnue ou à une extraction défailante.

Dans un contexte étudiant, les expériences ne se limitent pas à des emplois antérieurs. Les stages, travaux pratiques, projets personnels et réalisations universitaires peuvent documenter des compétences. Le prototype doit permettre de présenter ces éléments, tout en évitant de leur attribuer automatiquement une valeur professionnelle identique. Cette distinction motive le recours à une aide à l'examen des profils plutôt qu'à une décision automatique d'acceptation ou de refus.

1.2.2 Trois points de vue sur le même processus

Pour l'étudiant, la plateforme doit rendre possible la constitution d'un profil compréhensible et la consultation d'offres pertinentes. Le dépôt d'un CV facilite la saisie, mais ne dispense pas de vérifier les informations extraites. La possibilité d'ajouter un texte manuel devient particulièrement utile lorsque le fichier est difficile à lire. Le candidat a besoin d'un résultat interprétable : une compétence non détectée doit pouvoir

être distinguée d'une compétence qu'il ne possède pas.

Pour le recruteur, l'intérêt du classement est de structurer l'ordre de consultation des candidatures. Une liste triée réduit l'effort nécessaire pour repérer les profils partageant des caractéristiques avec une offre. Elle ne remplace ni la lecture du CV ni l'évaluation menée lors d'un entretien. Le résultat doit conserver les éléments qui permettent de contester une proposition du moteur : compétences communes, compétences manquantes et origine du calcul.

Pour l'administration de la plateforme, les enjeux concernent la cohérence des comptes, des offres et des candidatures, ainsi que la continuité du fonctionnement. Une architecture comprenant plusieurs composants peut rencontrer des indisponibilités partielles. La séparation du calcul en Python crée ainsi une exigence supplémentaire : l'application doit savoir reconnaître l'indisponibilité du service et fournir un comportement compréhensible. Le repli local répond à cet objectif, à condition que sa signification métier reste maîtrisée.

1.3 Problématique et objectifs

1.3.1 Question centrale

La problématique du projet peut être formulée ainsi : comment concevoir une plateforme web capable de rapprocher des offres et des profils étudiants à partir de données structurées et de CV, tout en produisant un résultat explicable et en restant utilisable lorsqu'un composant de traitement est indisponible ? Cette question relie la modélisation métier, le traitement automatique du texte et la robustesse de l'intégration.

Le terme *compatibilité* doit être défini avec précision. Dans SmartRecruit, il désigne une proximité calculée entre les informations déclarées dans une offre et celles disponibles dans un profil. Il ne désigne pas une probabilité validée de réussite professionnelle. Une valeur de 80 sur 100 indique le résultat d'une formule pondérée ; elle ne signifie pas que le candidat possède 80 % de chances d'être recruté. Cette interprétation évite de transformer une mesure technique en promesse prédictive.

Plusieurs tensions traversent cette problématique. Une représentation simple facilite l'explication mais reconnaît moins de variations linguistiques. Un service spécialisé facilite l'évolution du traitement mais ajoute une dépendance à l'exécution. Un dictionnaire de compétences améliore certains rapprochements tout en imposant des choix de catégorisation. Le projet doit rendre ces compromis visibles, car ils influencent les recommandations autant que le choix d'un langage ou d'un framework.

1.3.2 Objectifs fonctionnels et techniques

Le premier objectif consiste à organiser un parcours cohérent entre création du profil, publication d'une offre et candidature. Les informations doivent être réutilisables : le texte d'un CV et les compétences déclarées alimentent plusieurs comparaisons sans imposer une nouvelle saisie à chaque offre. Les résultats doivent être rattachés aux objets métier concernés, afin que l'utilisateur puisse revenir au profil et au besoin qui ont produit le rapprochement.

Le deuxième objectif est de rendre le matching calculable et inspectable. La présence d'une compétence recherchée, la proximité du vocabulaire et le partage d'une catégorie technique constituent des signaux distincts. Les combiner permet de produire un ordre de consultation, tandis que leur restitution aide à comprendre cet ordre. L'intérêt scientifique du prototype réside notamment dans la possibilité de relier chaque résultat à des opérations identifiées dans le code.

Le troisième objectif porte sur l'intégration d'un service Python avec l'application PHP. Cette séparation constitue un point d'extension pour les traitements de documents et les futurs modèles de langage. Un moteur local fournit une continuité lorsque le service distant n'est pas exploitable. Cependant, la simple présence de deux implémentations n'établit pas leur équivalence : celle-ci doit être vérifiée sur les données d'entrée, la normalisation, la formule et le résultat retourné.

Le quatrième objectif est de documenter les conditions de validité du système. Une extraction partielle, une compétence absente du dictionnaire ou une différence entre moteurs peuvent influencer le classement. Décrire ces situations fait partie de la qualité du logiciel. La validation doit ainsi couvrir les comportements attendus et les cas qui révèlent une limite, sans confondre un test de fonctionnement réussi avec la démonstration d'une pertinence générale en recrutement.

1.4 État de l'art et approches de rapprochement

1.4.1 Gestion des candidatures et recommandation

La gestion des candidatures et la recommandation répondent à deux besoins distincts. La première organise les informations et les transitions d'un processus : création d'une offre, réception d'une candidature, consultation puis décision. La seconde propose un ordre ou une sélection à partir de critères. Une application peut gérer correctement des candidatures sans disposer d'un moteur de recommandation pertinent ; réciproquement, une bonne mesure de similarité ne constitue pas une plateforme complète.

Burke distingue plusieurs familles de systèmes de recommandation, notamment

les approches fondées sur le contenu, le filtrage collaboratif et les approches fondées sur des connaissances. Son étude examine également des combinaisons hybrides de ces techniques [1]. Cette distinction permet de situer SmartRecruit : le rapprochement exploite le contenu des profils et des offres, enrichi par un dictionnaire et une taxonomie. Il n'utilise pas un apprentissage des préférences à partir d'historiques de comportements comparables.

Dans le périmètre du projet, aucune matrice d'interactions annotée n'est utilisée pour entraîner un modèle collaboratif. Une candidature enregistrée constitue un événement métier, mais elle ne devient pas automatiquement un exemple fiable de pertinence : un étudiant peut postuler par exploration, et l'absence de candidature peut simplement traduire une offre non consultée. Le choix d'un moteur fondé sur le contenu permet de calculer un résultat dès qu'une offre et un profil disposent d'une description, sans supposer un historique d'usage déjà constitué.

1.4.2 Représentation lexicale par TF-IDF et cosinus

Les représentations vectorielles classiques attribuent à chaque terme un poids décrivant sa contribution à un document. Les travaux de Salton et Buckley étudient les choix de pondération des termes pour la recherche documentaire [20]. TF-IDF associe une fréquence dans le document à une pondération liée à la diffusion du terme dans un corpus. Le cosinus compare ensuite l'orientation des vecteurs. Manning, Raghavan et Schütze présentent ces notions dans le cadre du modèle vectoriel de recherche d'information [9].

Pour SmartRecruit, cette approche offre un calcul facile à inspecter : les dimensions correspondent à des tokens issus des textes. Un rapprochement peut être relié à des termes communs, même si leur contribution n'est pas détaillée individuellement dans l'interface actuelle. La méthode ne nécessite aucun entraînement neuronal. Elle exige néanmoins des décisions de prétraitement, car le résultat dépend directement de ce qui est conservé, supprimé ou remplacé avant la vectorisation.

La variante du prototype calcule l'IDF sur deux documents seulement : l'offre et le profil comparés. Elle constitue donc une adaptation locale du principe TF-IDF, et non un index statistique construit sur l'ensemble des CV. Cette décision simplifie l'exécution mais réduit la diversité des fréquences documentaires possibles. Les conséquences mathématiques et les formules exactes sont exposées au chapitre 5.

1.4.3 Word2Vec : apprendre des représentations de mots

Mikolov, Chen, Corrado et Dean proposent en 2013 des architectures permettant d'apprendre des représentations vectorielles continues de mots, notamment les

modèles Continuous Bag-of-Words et Skip-gram [11]. Les relations entre mots sont apprises à partir de contextes observés dans un corpus. Cette approche permet de dépasser l'identité stricte des chaînes de caractères : des mots distincts peuvent obtenir des représentations proches. Une représentation statique associe toutefois un même vecteur à un mot, indépendamment de chacun de ses contextes d'emploi.

Une intégration à SmartRecruit demanderait de choisir le corpus ou les vecteurs préentraînés, puis une règle pour représenter un CV complet à partir de ses mots. Une simple moyenne risquerait de diluer une exigence technique importante dans le reste du document. L'amélioration ne pourrait donc pas être déduite du seul remplacement de TF-IDF par des vecteurs denses. Aucun modèle Word2Vec n'est chargé, entraîné ou évalué dans l'implémentation étudiée.

1.4.4 BERT et contextualisation du langage

BERT, présenté par Devlin, Chang, Lee et Toutanova, repose sur des représentations bidirectionnelles préentraînées à partir de texte non annoté, puis adaptées à différentes tâches de traitement du langage [2]. Les représentations tiennent compte du contexte gauche et droit, ce qui les distingue des vecteurs statiques de mots. Le modèle fournit ainsi un cadre pour exploiter des relations linguistiques plus riches que la seule cooccurrence de tokens dans deux documents.

Dans le projet, une telle évolution introduirait de nouveaux choix : segmentation des documents longs, représentation des sections du CV et adaptation éventuelle à la tâche de rapprochement. Il faudrait également déterminer comment convertir la sortie du modèle en explication utile au recruteur. La présence du nom BERT dans un dictionnaire de compétences ne signifie pas que BERT est exécuté : dans SmartRecruit, ce nom peut simplement être reconnu comme indice d'une compétence en traitement du langage.

1.4.5 CamemBERT et prise en compte du français

Martin et ses coauteurs présentent CamemBERT comme un modèle de langue pour le français fondé sur les Transformers. Leur étude évalue notamment l'étiquetage morphosyntaxique, l'analyse en dépendances, la reconnaissance d'entités nommées et l'inférence en langue naturelle [10]. Ces travaux fournissent une référence pour envisager une représentation linguistique adaptée au français. Leurs résultats concernent les tâches et jeux de données de l'article ; ils ne constituent pas une mesure de performance sur les CV de SmartRecruit.

Le choix d'un modèle francophone pourrait être pertinent pour des descriptions de formation et d'expérience rédigées en français. Il resterait cependant à traiter les

noms de technologies, souvent anglophones, ainsi que les documents mélangeant plusieurs langues. Le projet ne dispose pas d'une expérimentation permettant de choisir CamemBERT sur ce fondement. Il s'agit d'une piste d'évolution à comparer au moteur existant dans un protocole commun.

1.4.6 Sentence-BERT et similarité entre textes

Reimers et Gurevych proposent Sentence-BERT, une adaptation de BERT utilisant des structures siamoises et triplets pour produire des représentations de phrases comparables par similarité cosinus [19]. Cette construction vise la comparaison sémantique entre textes et permet de calculer des représentations séparées avant leur comparaison. Elle est donc conceptuellement plus proche d'un besoin de recherche par similarité que l'emploi direct d'un modèle BERT sans adaptation particulière.

Pour SmartRecruit, cette piste pourrait conduire à indexer séparément offres et profils, puis à comparer leurs vecteurs. Elle ne résoudrait pas à elle seule les différences entre une compétence obligatoire, une préférence et une expérience simplement mentionnée. Une forte proximité linguistique pourrait subsister malgré l'absence d'une exigence déterminante. Le maintien d'indicateurs explicites de couverture des compétences resterait alors un choix de conception à évaluer, et non une conséquence automatique du modèle.

1.5 Positionnement retenu pour le prototype

1.5.1 Une combinaison explicable de signaux

SmartRecruit retient un moteur hybride au sens opérationnel : plusieurs signaux calculés sont combinés dans une formule. La couverture des compétences mesure une correspondance explicite. Le cosinus fournit une proximité lexicale pondérée. La taxonomie valorise le partage de familles techniques. Dans le moteur PHP, un bonus supplémentaire dépend de l'expérience déclarée ou extraite. Le mot *sémantique* désigne ici principalement l'organisation manuelle d'alias et de catégories ; il ne doit pas être assimilé à une compréhension générale du CV.

Cette approche convient à l'objectif d'un produit minimum viable dont le comportement peut être étudié à partir du code. Elle rend les dépendances du résultat accessibles : une modification d'alias, de catégorie ou de coefficient peut être expliquée et testée. En contrepartie, la qualité du dictionnaire et des règles devient centrale. L'explicabilité de la formule ne garantit ni l'exactitude des informations extraites ni la pertinence du classement pour toutes les spécialités.

Le tableau 1.1 synthétise les options selon leur rôle possible dans le projet. Il présente une comparaison conceptuelle, sans classement expérimental des méthodes. Les coûts matériels, les temps de réponse et la pertinence ne sont pas chiffrés, car aucune campagne commune de mesure n'a été réalisée sur ces alternatives.

Table 1.1: Positionnement des familles de méthodes pour SmartRecruit

Approche	Apport envisagé	Point à vérifier dans le projet
Règles et alias	Rapprochements techniques explicitement définis	Couverture, ambiguïtés et maintenance du dictionnaire
TF-IDF et cosinus	Similarité fondée sur le vocabulaire observé	Qualité des tokens et choix du corpus IDF
Word2Vec	Proximité apprise entre mots	Agrégation des mots et adéquation du corpus
BERT / CamemBERT	Représentations dépendant du contexte	Adaptation à la tâche et traitement des CV longs
Sentence-BERT	Comparaison de représentations de textes	Pertinence métier et conservation des exigences explicites

1.5.2 Explicabilité, responsabilité et limites des preuves

L'application du classement à des personnes exige de distinguer la visibilité des règles de leur justification. Raghavan, Barocas, Kleinberg et Levy analysent les revendications et pratiques de fournisseurs d'évaluations algorithmiques de recrutement, notamment leurs choix de données, de cibles et de validation [18]. Cette référence invite à examiner ce qu'un score mesure effectivement et les preuves qui soutiennent son usage. Elle ne permet pas d'affirmer que SmartRecruit supprime des biais de recrutement.

Dans le prototype, un vocabulaire limité aux technologies recensées favorise mécaniquement les profils qui emploient ces expressions. La longueur du texte, la qualité de l'extraction et la manière de présenter les projets influencent également les entrées du calcul. Ces facteurs sont des conséquences de conception identifiables ; leur effet sur des populations ne peut pas être quantifié sans données et protocole adaptés. Le mémoire ne présente donc aucune mesure d'équité, aucune validation psychométrique et aucune estimation de réussite professionnelle.

La contribution du projet est située au niveau de l'ingénierie logicielle : intégrer un parcours de recrutement, un traitement de CV et un calcul explicable dans une architecture étudiable. Les exemples numériques servent à vérifier le calcul. Les sondes ciblées du chapitre 6 servent à observer des comportements précis. Une évaluation scientifique de pertinence exigerait, en complément, des couples offre-profil annotés,

des critères d'évaluation établis et une comparaison reproductible entre méthodes.

Ce positionnement permet de développer les choix d'architecture et de réalisation sans surinterpréter le terme d'intelligence artificielle. Le moteur constitue une base de référence explicable et un point d'extension pour des expérimentations ultérieures. Ses limites actuelles définissent les questions à traiter avant de lui confier un rôle plus important dans une décision de recrutement.

CHAPTER 2

Analyse et spécification des besoins

Ce chapitre établit le contrat fonctionnel de SmartRecruit et les conditions dans lesquelles son fonctionnement peut être évalué. Le projet porte sur la mise en relation d'étudiants et de recruteurs, avec une assistance au rapprochement entre les profils et les offres. Il est présenté dans le cadre du mémoire de Master en Développement Logiciel de Rahma Amiri à l'Institut Supérieur d'Informatique de Tunis El Manar, associé à un stage chez Best Solutions du 2 mars au 2 juillet 2026, sous l'encadrement professionnel de Hammami Housseem. Ces éléments situent le travail ; ils ne permettent pas, à eux seuls, de reconstituer des entretiens, des décisions d'entreprise ou un historique de développement.

La spécification repose donc sur les parcours visibles dans le dépôt, les contrôles des routes, le modèle de données et la documentation disponible. Elle distingue systématiquement trois catégories : une capacité effectivement présente, une exigence nécessaire pour rendre cette capacité fiable et une évolution proposée. Un critère d'acceptation exprime un résultat à vérifier ; sa présence dans ce chapitre ne constitue pas une preuve de réussite d'un test.

2.1 Problématique et périmètre fonctionnel

Un dossier de candidature réunit des informations structurées, comme une adresse électronique ou une liste de compétences, et un document largement non structuré, le curriculum vitae. De son côté, une offre décrit un poste ou un stage au moyen d'un intitulé, d'un texte libre et de compétences recherchées. Le rapprochement de ces informations exige de reconnaître certaines variantes de vocabulaire et de rendre les résultats comparables. Une simple liste chronologique de candidatures ne répond pas à ce besoin d'orientation.

SmartRecruit organise cette relation autour de quatre objets métier : le compte, le profil étudiant, l'offre et la candidature. L'analyse du CV complète le profil ; le calcul de compatibilité relie un profil à une offre ; le suivi de candidature représente une démarche explicite de l'étudiant. Cette dernière distinction est essentielle. Un profil peut être recommandé pour une offre sans avoir candidaté. Le classement proposé au recruteur ne doit donc pas être présenté comme une liste de personnes ayant manifesté

leur intérêt lorsque le calcul porte sur l'ensemble du vivier.

Le périmètre du MVP comprend l'inscription d'étudiants et de recruteurs, l'authentification par mot de passe, la modification des profils, le dépôt de CV, la création et la modification d'offres, la candidature en un clic, le classement des profils et le changement d'état des candidatures. Les interfaces affichent des informations adaptées au rôle connecté. Le projet comporte également une vue d'administration et un point de diagnostic de la communication avec le service Python.

Le périmètre ne comprend pas une chaîne complète de recrutement jusqu'à la contractualisation. Aucun module de visioconférence, de messagerie entre utilisateurs, de calendrier d'entretiens, de notification automatique ou de signature n'est établi par le code étudié. L'état `interview` représente une catégorie de suivi, pas une réservation de rendez-vous. De même, une valeur numérique de compatibilité constitue une aide à l'examen des dossiers ; elle ne démontre ni la qualité future du travail d'une personne ni la probabilité de réussite de son recrutement.

2.2 Acteurs et responsabilités

L'acteur principal du parcours candidat est l'étudiant. Il crée un compte, décrit son profil, apporte son CV, consulte les offres publiées et suit ses candidatures. Le recruteur constitue l'autre acteur principal : il publie des offres, consulte les profils accessibles, examine les classements et renseigne les états de candidature associés à ses offres. L'administrateur dispose d'une visibilité plus large et peut intervenir sur les profils et les offres. Le visiteur accède à l'accueil et aux formulaires d'entrée dans le système.

Le service d'analyse Python intervient comme système secondaire sollicité par l'application. Il n'est pas un utilisateur authentifié de l'interface et ne décide pas de l'autorisation d'une opération métier. La base de données et le stockage de fichiers sont, quant à eux, des moyens internes de persistance. Cette séparation permet de ne pas confondre les droits d'une personne, les échanges entre applications et la localisation physique des informations.

La figure 2.1 présente les interactions principales. La notation UML facilite la distinction entre acteurs et services rendus par le système [12]. Ce diagramme constitue une vue synthétique ; les contrôles portant sur la propriété d'un objet, les erreurs de validation et les conséquences d'une indisponibilité sont détaillés dans les scénarios suivants.

2.3 Spécification des cas d'utilisation

Table 2.1: Responsabilités principales des acteurs

Acteur	Actions attendues	Limite de responsabilité
Étudiant	Compléter son profil et candidater.	Ne pas modifier un autre profil.
Recruteur	Gérer ses offres et leur suivi.	Ne pas décider pour les offres d'un autre recruteur.
Administrateur	Consulter la synthèse et gérer les objets autorisés.	Ne pas confondre administration et validation du score.
Visiteur	Consulter l'accueil, s'inscrire, se connecter.	Aucun accès implicite aux opérations protégées.
Service Python	Extraire du texte et calculer un rapprochement.	Aucun accès direct aux décisions métier.

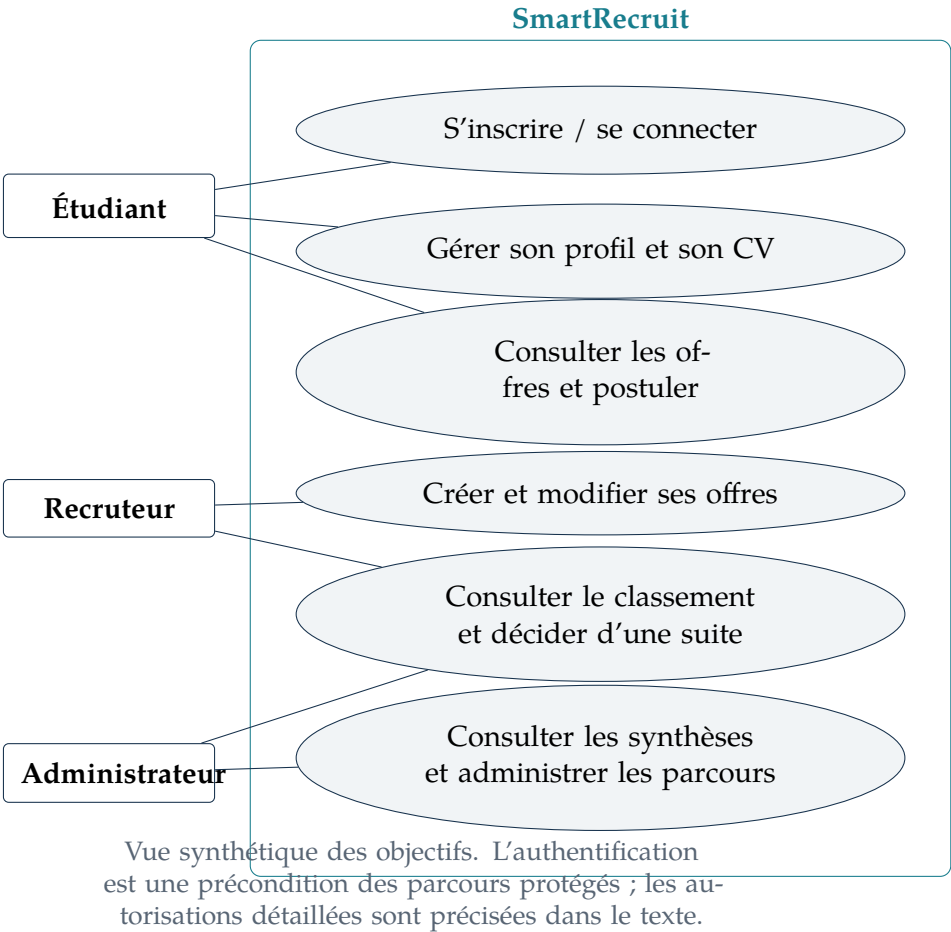


Figure 2.1: Vue synthétique des cas d'utilisation de SmartRecruit

2.3.1 UC01 : créer un compte et ouvrir une session

Préconditions. L'utilisateur n'a pas besoin d'être connecté pour accéder à l'inscription. La base SQL doit être disponible, car les routes d'inscription et de connexion refusent ces opérations lorsque ce stockage est inaccessible. L'inscription publique accepte les rôles étudiant et recruteur ; elle ne propose pas de créer un administrateur.

Scénario nominal. L'utilisateur choisit son rôle, fournit son identité de connexion, renseigne les informations requises pour son type de profil et confirme son mot de passe. Laravel vérifie les champs, notamment le format et l'unicité de l'adresse électronique, la longueur minimale du mot de passe et sa confirmation. Le compte et le profil associé sont créés ensemble dans une transaction. Le système place ensuite les informations nécessaires dans la session et présente le tableau de bord du rôle retenu. Lors d'une connexion ultérieure, il recherche le compte puis vérifie le mot de passe à partir de son empreinte.

Alternatives et postconditions. Un formulaire invalide ne doit pas créer de compte partiel. Une adresse déjà utilisée doit produire une erreur exploitable, sans attribuer le compte existant à une nouvelle personne. Une connexion échouée conserve un message générique. La session ouverte identifie un seul utilisateur et son rôle. Les routes actuelles limitent la fréquence des demandes de connexion et d'inscription ; la régénération de l'identifiant de session et l'invalidation complète à la déconnexion restent des exigences de durcissement, conformément aux recommandations relatives aux sessions [14].

2.3.2 UC02 : compléter le profil et déposer un CV

Préconditions. L'étudiant est connecté et agit sur le profil associé à son compte. L'administrateur dispose d'un parcours distinct pour créer ou modifier un profil. Un recruteur peut consulter les profils accessibles, mais ne bénéficie pas de l'autorisation de les modifier.

Scénario nominal. L'étudiant renseigne un intitulé, sa formation, son expérience, ses compétences et, éventuellement, des liens professionnels. Il fournit un fichier ou du texte de CV. Le formulaire vérifie les types et les longueurs ; les formats annoncés par la validation sont PDF, TXT et Markdown, avec une limite de 4 Mo. Le parseur tente d'extraire le texte et des informations utiles. Les compétences déclarées et détectées peuvent être combinées, puis le profil est enregistré avec ses métadonnées. La page de détail permet de relire le résultat et de corriger les informations.

Alternatives et postconditions. Un document peu lisible ne doit pas être transformé silencieusement en profil complet. Le code prévoit un avertissement lorsque l'extraction paraît insuffisante et invite à compléter le texte manuellement. Cette pos-

sibilité est indispensable pour un PDF composé d'images, puisque l'OCR n'est pas établi dans le MVP. Le contrôle d'extension et de taille constitue une première barrière, mais ne garantit pas, à lui seul, un traitement sûr de tout document [15]. La postcondition attendue est un profil cohérent avec les informations réellement conservées, accompagné d'un état explicite de l'extraction.

2.3.3 UC03 : publier et maintenir une offre

Préconditions. Le recruteur ou l'administrateur est connecté. Pour une modification par un recruteur, l'offre doit appartenir à ce compte. L'offre comprend au minimum un intitulé, une entreprise et une description ; les compétences recherchées peuvent être saisies séparément.

Scénario nominal. L'acteur complète le formulaire, soumet les informations et reçoit la confirmation de création. Dans l'état actuel, une offre nouvelle est publiée par défaut. Le formulaire de modification permet ensuite de choisir un état parmi `draft`, `published` et `closed`. Le recruteur retrouve ses offres dans une liste filtrable par état et peut isoler celles qui ont reçu des candidatures. Les étudiants disposent d'une liste limitée aux offres publiées.

Alternatives et postconditions. Une offre inexistante donne lieu à une réponse d'absence ; la tentative de modification d'une offre appartenant à un autre recruteur est refusée. Une validation incorrecte préserve les données déjà enregistrées. La fermeture d'une offre doit signifier qu'aucune nouvelle candidature ne peut être reçue. Cette dernière règle constitue une exigence à compléter : le contrôle existe pour la consultation d'une offre par un étudiant, mais la route de candidature ne revalide pas explicitement son état.

2.3.4 UC04 : candidater à une offre

Préconditions. L'étudiant possède une session et un profil valides. L'offre existe et, selon la règle métier attendue, son état est publié. La candidature désigne le profil de la session ; son identité ne doit pas être choisie librement dans une valeur envoyée par le navigateur.

Scénario nominal. L'étudiant ouvre une offre, déclenche l'action de candidature et reçoit un message de confirmation. Le système relie l'offre au profil, affecte l'état initial `submitted` et enregistre la date de candidature. Le tableau de bord étudiant permet ensuite de retrouver cette démarche et son état. Le dépôt d'un nouveau CV relève du parcours de mise à jour du profil ; aucun formulaire distinct de candidature avec création simultanée de profil ne doit être supposé sur la base d'une ancienne description documentaire.

Alternatives et postconditions. Une seconde demande concernant le même couple doit rester sans effet sur le nombre de candidatures. Le stockage SQL possède une contrainte unique pour ce couple et la méthode de candidature vérifie préalablement son existence. Cependant, deux requêtes simultanées peuvent franchir cette vérification ; il reste nécessaire de convertir le conflit SQL en réponse métier stable. Après réussite, une candidature unique relie les deux objets, sans modifier l'identité du candidat ni les caractéristiques de l'offre.

2.3.5 UC05 : examiner un classement explicable

Préconditions. Le recruteur consulte une offre qui lui appartient, ou l'administrateur ouvre une offre accessible. Des profils sont disponibles. Une candidature préalable n'est pas nécessaire au calcul d'une recommandation, puisque les classements actuels peuvent porter sur tout le vivier.

Scénario nominal. Le système rapproche la description de l'offre et chaque profil, attribue un score, produit des listes de compétences reconnues ou manquantes, puis trie les résultats. Il utilise le service Python lorsqu'il obtient une réponse exploitable ; sinon, le moteur PHP fournit un résultat local. Les recommandations d'offres à destination des étudiants suivent le raisonnement symétrique : un profil est comparé aux offres publiées.

Alternatives et postconditions. Une indisponibilité du service ne doit pas provoquer une présentation trompeuse de l'origine du calcul. Le classement doit identifier la méthode utilisée et distinguer un texte inexploitable d'une faible compatibilité réellement calculée. L'absence de compétences détectées ne prouve pas leur absence chez la personne. Dans le MVP, un score peut être enregistré lorsqu'une candidature existe et provoquer une présélection automatique. L'exigence cible consiste à séparer cette assistance de la décision humaine et à rendre toute transition automatique explicite et traçable.

2.3.6 UC06 : suivre et qualifier une candidature

Préconditions. La candidature existe. Le recruteur est propriétaire de l'offre associée, ou l'acteur est administrateur. L'état demandé appartient à la liste reconnue : soumise, présélectionnée, entretien ou rejetée.

Scénario nominal. Le recruteur examine le dossier et son explication de score, choisit l'état approprié et soumet sa décision. Le système contrôle le rôle, l'appartenance de l'offre et la valeur demandée, puis actualise la candidature. L'étudiant retrouve cet état dans son espace. Le terme présélection signifie une étape intermédiaire ; aucun état de recrutement définitif n'existe dans le schéma étudié.

Alternatives et postconditions. Une tentative sur la candidature d'un autre recruteur est refusée. Une valeur non reconnue est rejetée. La conception cible devra aussi définir les transitions autorisées entre états et conserver l'auteur, la date et la raison éventuelle d'un changement. Le code actuel valide une liste de valeurs, sans machine à états ni journal des décisions ; l'historique complet ne peut donc pas être présenté comme une fonctionnalité acquise.

2.4 Exigences identifiées et critères d'acceptation

Les exigences du tableau 2.2 servent de repères pour relier les parcours, la conception et une future campagne de tests. Le mot attendu désigne une obligation de résultat à vérifier. Une fonctionnalité visible peut satisfaire une partie du critère sans satisfaire ses cas limites, notamment en présence de concurrence, d'un fichier inattendu ou d'un service indisponible.

Table 2.2: Exigences fonctionnelles et observations vérifiables

Identifiant	Exigence	Critère d'acceptation attendu
EF01	Authentifier les comptes.	Des identifiants valides ouvrent le bon espace ; les identifiants invalides sont refusés.
EF02	Limiter les modifications au propriétaire.	Un étudiant ne peut modifier un autre profil ; un recruteur ne peut modifier une offre tierce.
EF03	Permettre la correction du CV.	Les champs corrigés sont restitués après sauvegarde ; une extraction insuffisante est signalée.
EF04	Gérer la publication.	Une offre fermée ou en brouillon ne reçoit aucune nouvelle candidature étudiante.
EF05	Assurer l'unicité des candidatures.	Deux soumissions, même simultanées, conservent un seul couple offre/profil.
EF06	Expliquer les rapprochements.	Le résultat indique un score, les éléments de rapprochement et la source du calcul.
EF07	Contrôler le suivi.	Seul un acteur habilité change l'état ; une valeur inconnue est refusée.

Identifiant	Exigence	Critère d'acceptation attendu
EF08	Distinguer vivier et candidatures.	L'interface permet d'identifier si un profil classé a effectivement candidaté.

Les exigences non fonctionnelles concernent d'abord la confidentialité des CV et l'intégrité des décisions. Une route protégée par rôle ne suffit pas lorsque l'opération vise un objet individuel : il faut aussi vérifier le droit sur cet objet. La validation serveur doit s'appliquer à chaque entrée, même si le formulaire la contrôle déjà dans le navigateur [7]. Les fichiers doivent disposer de règles d'accès et d'un cycle de conservation cohérents avec les profils auxquels ils se rapportent.

La disponibilité doit être définie par fonction. Un moteur de score local peut maintenir une aide au classement en cas de panne Python. En revanche, une bascule automatique de MySQL vers un fichier de démonstration ne garantit pas la continuité des données métier. L'exigence cible est de rendre cette indisponibilité explicite et de ne confirmer une écriture que dans le stockage de référence. Elle ne se confond pas avec l'objectif pédagogique de lancer facilement une démonstration.

La performance doit être mesurée sur un volume décrit de profils, d'offres et de documents, avec une configuration matérielle connue. Aucun temps de réponse garanti n'est déduit du dépôt. Les critères à construire portent sur la durée de l'extraction, la latence d'un classement et le nombre de requêtes par page. La maintenabilité sera appréciée par la séparation des responsabilités, la stabilité des contrats entre PHP et Python et la possibilité de tester les règles sans lancer tout le système.

Table 2.3: Exigences de qualité à vérifier lors de la consolidation

Identifiant	Qualité attendue	Critère d'acceptation proposé
ENF01	Confidentialité	Une demande sans autorisation ne permet pas d'obtenir un document privé.
ENF02	Intégrité	Une modification interrompue ne laisse pas un profil partiellement mis à jour.
ENF03	Disponibilité maîtrisée	Une panne SQL est annoncée ; aucune écriture métier n'est confirmée dans un stockage de démonstration.
ENF04	Diagnostic	Un échec réseau est distingué d'un document dont le texte est inexploitable.
ENF05	Performance mesurable	Chaque mesure précise volume, configuration, méthode et distribution des durées observées.
ENF06	Reproductibilité	Des entrées identiques et une même version de moteur donnent un résultat vérifiable.

Le tableau 2.3 complète les exigences fonctionnelles sans attribuer au prototype

des garanties non mesurées. Il permet de préparer des essais d'échec et de reprise, au même titre que les parcours nominaux. Les seuils quantitatifs de performance restent à fixer à partir d'un contexte d'utilisation explicite et d'une première mesure reproductible.

2.5 Contraintes, priorités et démarche rétrospective

Le projet utilise PHP et Laravel pour l'application, Python pour l'analyse, MySQL pour la persistance principale et Blade pour les vues. Ces choix constituent des contraintes de continuité : une évolution doit préserver les parcours existants et éviter de disperser davantage les règles. Le délai de stage fixe par ailleurs un cadre académique borné. Il justifie un MVP démontrable, mais pas la présentation comme achevées de fonctions absentes ou d'essais non documentés.

La priorité de consolidation porte sur les règles qui empêchent une action incorrecte : accès aux objets, état d'une offre au moment de candidater, unicité en concurrence, cohérence d'une modification de profil et séparation entre score et décision. Viennent ensuite la qualité de l'information extraite et la maîtrise du coût des classements. L'introduction d'un modèle linguistique plus complexe appartient aux évolutions possibles ; elle ne résout pas ces questions de fonctionnement et de traçabilité.

La démarche présentée ici est rétrospective. Elle ne prétend pas reconstituer des sprints, des réunions ou des validations par l'entreprise dont aucune trace n'a été examinée. Elle organise les éléments disponibles en quatre ensembles de travail : identification des parcours, formalisation des règles, confrontation avec le code et définition des écarts à vérifier. Pour chaque exigence, un lien peut ainsi être établi avec un cas d'utilisation, un composant technique et un scénario de contrôle. Cette organisation rend le mémoire exploitable même lorsque l'historique détaillé des décisions n'est pas disponible.

Le chapitre suivant traduit ces besoins en structures applicatives et en relations de données. Il décrit d'abord l'architecture réellement présente, puis formule une architecture cible. Cette distinction garantit que les choix proposés restent évaluables et que la conception ne masque pas les limites observées du MVP.

CHAPTER 3

Conception et architecture de la solution

La conception de SmartRecruit doit répondre à deux questions complémentaires : comment les fonctions du MVP sont-elles réparties dans le système existant, et quelles modifications rendraient cette organisation plus fiable ? Le présent chapitre décrit d’abord les composants et les structures effectivement présents dans le dépôt. Il expose ensuite les écarts entre les règles métier du chapitre 2 et les garanties techniques actuelles. Les éléments qualifiés de cibles ou de propositions ne correspondent pas à des développements déjà réalisés.

3.1 Architecture générale observée

L’application comporte trois ensembles principaux : un serveur Laravel, un service Python et une base MySQL. Le navigateur communique avec Laravel pour consulter les pages et soumettre les formulaires. Laravel gère la session, contrôle les rôles, valide les entrées, accède aux données et prépare les vues Blade. Il appelle le service Python pour certaines opérations d’extraction et de rapprochement. Python retourne des résultats de calcul ; il ne manipule pas directement les comptes ou les candidatures en base.

Cette organisation introduit une frontière de service claire pour l’analyse de documents, mais ne constitue pas un découpage de tout le domaine métier en microservices. Le recrutement reste concentré dans une application Laravel. Il est donc plus exact de décrire un noyau applicatif avec un service spécialisé que d’attribuer au projet toutes les propriétés d’une architecture de microservices. La littérature souligne que la séparation en services implique aussi des responsabilités d’exploitation et des contrats entre composants, au-delà de l’utilisation de processus distincts [8].

La figure 3.1 situe les flux et les principaux stockages. Les CV sont conservés dans un répertoire de fichiers, tandis que leurs références et informations d’analyse sont enregistrées en base. Le fichier JSON de démonstration représente une solution de repli présente dans le code. Il doit être distingué de MySQL, car les deux supports ne forment ni une réplique ni un système de synchronisation bidirectionnelle.

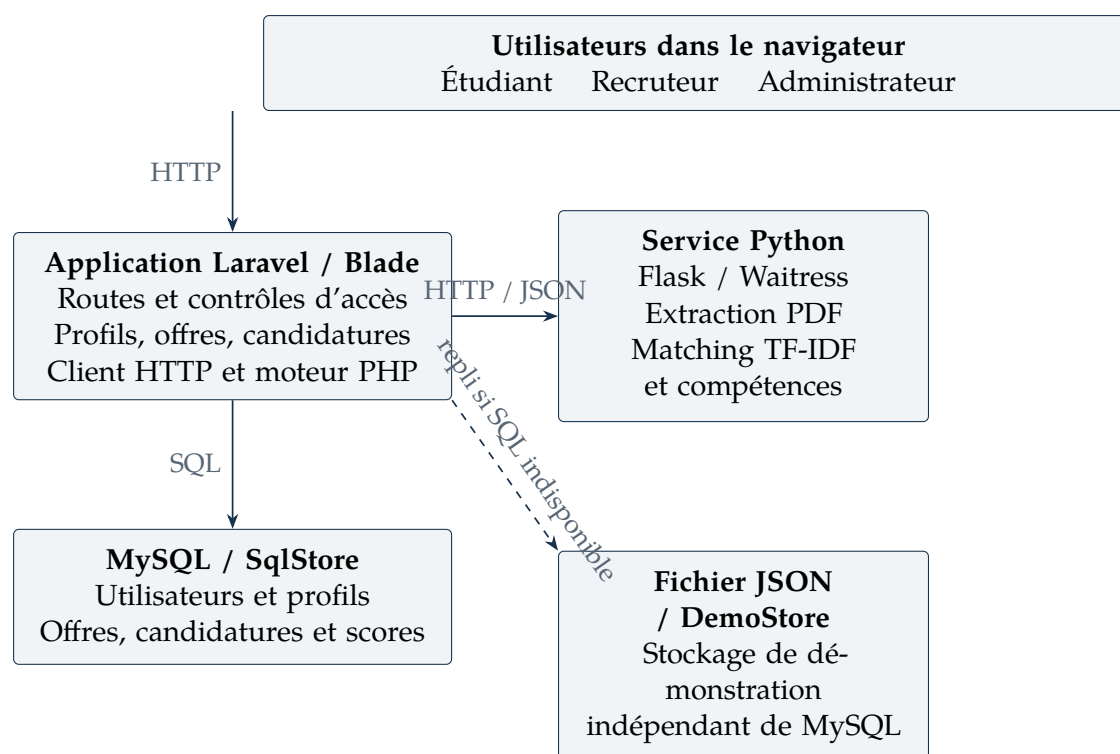


Figure 3.1: Architecture applicative observée et échanges entre composants

Le fichier de composition Docker déclare les services web, IA et MySQL, leur réseau et les volumes nécessaires. Il fournit une description de lancement distribuée et reproductible en principe [3]. Son existence ne prouve toutefois pas que le déploiement a été exécuté avec succès dans l'environnement étudié. Les contrôles de disponibilité, les sauvegardes, les paramètres de production et la gestion des secrets doivent être examinés séparément ; ils ne sont pas déduits d'un diagramme d'architecture.

3.2 Organisation du noyau Laravel

Les routes et une partie importante de l'orchestration métier sont réunies dans `routes/web.php`. Les fonctions auxiliaires sélectionnent le stockage, construisent les textes de comparaison, calculent les classements et fournissent l'utilisateur courant. Des fermetures de routes enchaînent validation, contrôle d'appartenance, appel aux classes de support et création de la réponse. Laravel autorise cette forme de définition des routes [6] ; le choix devient cependant plus coûteux à maintenir lorsque les mêmes règles interviennent dans plusieurs parcours.

La classe `SqlStore` regroupe les accès relationnels et convertit les lignes SQL en tableaux utilisés par les vues. Le projet emploie le constructeur de requêtes Laravel plutôt qu'un ensemble de modèles Eloquent représentant les entités. `DemoStore` propose des opérations proches sur un document JSON. Aucune interface commune ex-

plicite ne formalise leur contrat, et certaines fonctions, comme l'inscription ou la sauvegarde des scores, ne sont pas disponibles sur les deux supports. Le code appelant teste parfois la présence d'une méthode avant de l'utiliser.

Les autres classes de support ont des responsabilités plus spécialisées. `CvParser` coordonne l'extraction et construit les informations du profil. `AiClient` encapsule les appels au service Python. `SemanticMatcher` fournit le calcul local. Le middleware `EnsureRole` vérifie que le rôle de session appartient à la liste autorisée ; les contrôles concernant le propriétaire d'une offre ou d'un profil sont ensuite exécutés dans les routes. Cette répartition donne un point d'entrée identifiable à chaque responsabilité, tout en laissant plusieurs décisions transversales dispersées.

L'identité est stockée dans une valeur de session propre au projet. Elle comprend le rôle et, pour un étudiant, l'identifiant public du profil. Les routes ne doivent donc jamais assimiler la possession d'une session à un droit sur tous les objets. L'étudiant peut atteindre son propre profil, tandis qu'un recruteur est limité à ses offres pour leur modification et la qualification de leurs candidatures. La consultation du vivier est plus large : elle correspond à une fonctionnalité distincte, dont la politique de visibilité doit être assumée explicitement.

3.3 Modèle relationnel et cardinalités

3.3.1 Entités persistées

La migration principale définit sept tables préfixées par `sr_`. Le tableau 3.1 décrit leur rôle sans reproduire de données de comptes ou de CV. Les clés numériques assurent les jointures internes ; les identifiants publics sont utilisés dans les échanges applicatifs pour plusieurs entités. L'existence d'un identifiant public distinct ne constitue pas un contrôle d'accès : l'autorisation reste requise pour chaque opération.

Table 3.1: Tables du modèle relationnel existant

Table	Responsabilité	Informations principales
<code>sr_users</code>	Identité de connexion.	Identifiant public, nom, adresse électronique unique, rôle, empreinte du mot de passe.
<code>sr_student_profiles</code>	Description du candidat.	Référence utilisateur, formation, expérience, compétences, texte CV, métadonnées.

Table	Responsabilité	Informations principales
<code>sr_recruiter_profiles</code>	Contexte du recruteur.	Référence utilisateur, entreprise, fonction et site.
<code>sr_cv_documents</code>	Référence des documents.	Profil étudiant, nom, chemin, type, taille et résultat du parseur.
<code>sr_offers</code>	Description du besoin.	Recruteur, intitulé, entreprise, description, compétences et état.
<code>sr_applications</code>	Démarche de candidature.	Offre, profil étudiant, état et date de candidature.
<code>sr_match_scores</code>	Résultat associé à une candidature.	Score, composantes, compétences rapprochées ou manquantes et réponse détaillée.

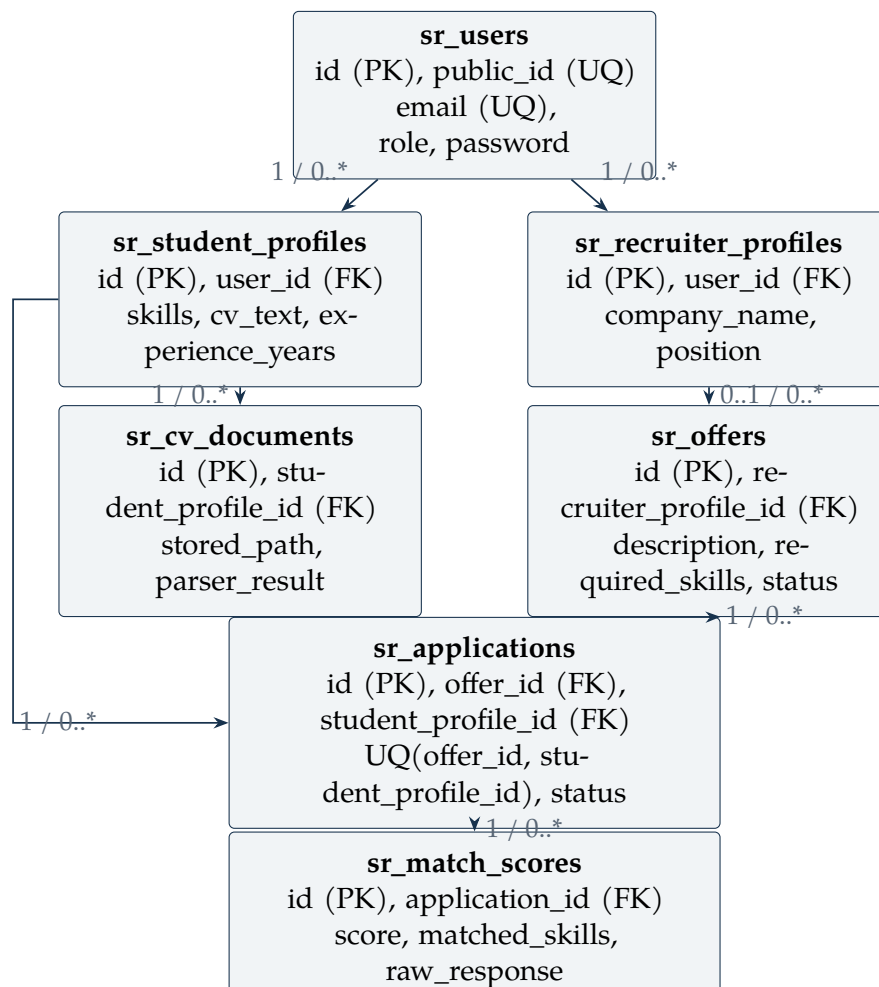
La figure 3.2 représente les relations principales. Les cardinalités doivent être lues en distinguant la règle métier attendue de ce que les contraintes SQL imposent réellement. Cette distinction est particulièrement importante pour les profils utilisateur et les résultats de matching.

3.3.2 Relations et garanties effectives

Chaque profil étudiant référence exactement un utilisateur, et chaque profil recruteur référence également un utilisateur. En revanche, la colonne de référence n'est pas unique dans ces tables. La base permet donc plusieurs profils du même type pour un utilisateur, alors que les parcours applicatifs supposent généralement un seul profil correspondant au rôle. Une cardinalité métier de zéro ou un profil par type nécessiterait une contrainte supplémentaire. Les clés étrangères ne vérifient pas non plus que le rôle du compte correspond au type de profil associé.

Un profil étudiant peut disposer de plusieurs documents CV. Chaque document appartient à un seul profil. Une offre référence au plus un profil recruteur : cette référence est nullable. Un recruteur peut être lié à plusieurs offres. Une candidature associe exactement une offre et un profil étudiant ; une contrainte unique interdit de répéter ce couple. Cette unicité exprime une règle métier importante à un niveau indépendant du formulaire et protège aussi contre les écritures concurrentes [13].

Une candidature peut posséder plusieurs lignes dans `sr_match_scores`, car aucune unicité n'est définie sur sa référence. Pourtant, le code recherche la dernière



Cardinalités autorisées par le schéma SQL fourni. PK : clé primaire ; FK : clé étrangère ; UQ : unicité. Un profil par utilisateur et un score courant par candidature sont des règles métier à renforcer.

Figure 3.2: Modèle de données des sept tables de SmartRecruit

ligne puis la met à jour, et n'ajoute une ligne que s'il n'en trouve pas. Le schéma permet donc un historique que l'implémentation ne construit pas de manière organisée. Il faut choisir entre un résultat courant unique et un historique versionné ; conserver une multiplicité implicite n'offre ni une garantie d'unicité ni une traçabilité complète.

Les suppressions d'utilisateurs entraînent celles des profils associés ; les documents et candidatures concernés bénéficient également de relations avec suppression en cascade. Les scores suivent la suppression de leur candidature. La suppression d'un profil recruteur met à null la référence portée par ses offres, au lieu de supprimer ces dernières. Ces règles sont inscrites dans le schéma, même si l'interface du MVP ne propose pas tous les parcours de suppression. Une future fonction d'effacement devra aussi traiter les fichiers, que les cascades SQL ne peuvent pas retirer du disque.

3.3.3 Champs structurés, JSON et évolution du schéma

Les attributs stables, comme les états et les références, sont représentés par des colonnes dédiées. Les listes de compétences, liens, métadonnées du CV et réponses de calcul utilisent des champs JSON. Cette combinaison évite de créer de nombreuses tables pour des informations variables dans un prototype. Elle limite en revanche la validation relationnelle fine et complique certaines recherches transversales, par exemple compter les profils associés à une compétence canonique avec une même règle de normalisation.

Les états d'offre sont bornés à brouillon, publié et fermé ; les états de candidature à soumise, présélectionnée, rejetée et entretien. Cette contrainte restreint les valeurs stockées, mais ne décrit pas les transitions autorisées. Les dates de création et de modification permettent de situer une ligne ; elles ne constituent pas un journal des opérations. De même, les colonnes prévues pour le type et la taille du CV sont présentes, mais les insertions du store renseignent actuellement une valeur nulle et zéro. La présence d'une colonne ne prouve donc pas que son information est exploitée.

La migration comporte aussi une adaptation des tables existantes, centrée sur l'ajout et le remplissage de l'identifiant public. Ce chemin ne reconstitue pas nécessairement les mêmes contraintes qu'une création neuve, notamment son unicité. La conception cible doit séparer création initiale et migrations d'évolution, avec vérification des doublons avant l'ajout d'un index. Une installation issue d'un schéma ancien doit être évaluée comme un cas particulier, sans supposer son équivalence automatique à une base neuve.

3.4 Contrats d'API et flux de traitement

3.4.1 Interface entre Laravel et Python

Le service Python expose trois opérations. GET/health indique que le service répond et annonce son algorithme. POST/match reçoit un objet JSON contenant les textes de l'offre et du CV ainsi que les listes de compétences requises et candidates. Il retourne le score et des informations explicatives. POST/parse-cv reçoit un fichier dans un formulaire multipart, extrait son texte et retourne notamment des compétences et des adresses détectées. L'absence du champ fichier donne lieu à une réponse d'erreur dédiée.

Ce contrat réalise une séparation utile : Laravel conserve les décisions d'accès et les relations métier, Python traite un contenu transmis. Le service d'analyse n'a pas besoin de connaître le mot de passe ou l'identifiant SQL d'un étudiant pour calculer un rapprochement. Une architecture cible devrait conserver cette minimisation des données et éviter de transmettre des informations sans utilité pour le traitement. La réponse de l'analyse doit, réciproquement, être considérée comme une entrée externe à valider avant toute persistance.

Les mécanismes d'appel ne sont pas uniformes dans le client actuel. L'extraction multipart utilise le client HTTP Laravel et un délai configuré de six secondes, tandis que les échanges JSON reposent sur un flux HTTP PHP avec un délai de 0,4 seconde. Ces valeurs sont des paramètres du code, pas des mesures de performance. Les erreurs conduisent généralement à une valeur nulle et à un repli local. La cible devrait distinguer indisponibilité réseau, réponse invalide, dépassement de délai et document inexploitable afin d'afficher un état compréhensible et d'établir un diagnostic utile.

3.4.2 Enchaînement du dépôt et du classement

La saisie du profil, la candidature et la consultation du classement constituent des actions distinctes. L'utilisateur soumet d'abord les informations à Laravel ; l'application contrôle leur recevabilité, sollicite l'analyse si nécessaire et conserve le profil. Une candidature peut ensuite relier ce profil à une offre. La figure 3.3 détaille la consultation du classement : Laravel compare chaque profil à l'offre, utilise le résultat Python lorsqu'il est exploitable, puis transmet à la vue le score et ses explications. Un résultat peut être conservé dans la table des scores si une candidature existe.

Dans l'implémentation présente, le résultat local est calculé avant l'éventuel appel au moteur distant. Des champs de la réponse distante remplacent ensuite ceux du calcul local. Le service est interrogé au cours des demandes web, sans file de traitement établie dans le dépôt. Cette conception facilite la lecture d'un parcours complet, mais lie la durée de la réponse au nombre de comparaisons demandées. Elle exige aussi de vérifier la cohérence des réponses des deux moteurs, car leur interchangeabilité ne

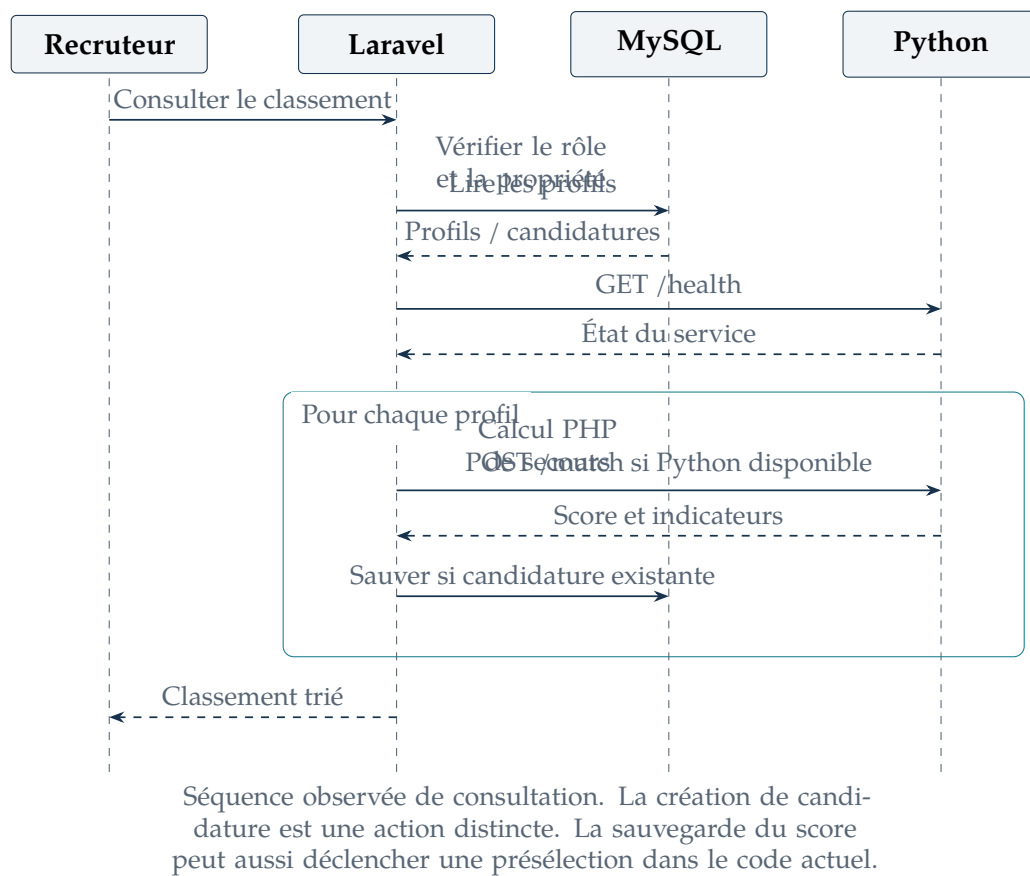


Figure 3.3: Séquence observée de consultation du classement d’une offre

résulte pas seulement d'un nom de méthode commun.

Une particularité mérite d'être explicitée : la consultation du classement peut écrire un score et faire évoluer une candidature vers `shortlisted` lorsque son score atteint 70. Une page de consultation possède donc un effet métier. Si une personne ouvre le classement après une modification de CV, elle peut déclencher un changement d'état sans action de décision séparée. La conception cible doit rendre les lectures sans effet sur le suivi, ou formaliser distinctement la règle d'automatisation avec un événement identifiable et une politique de transition.

3.5 Transactions, concurrence et cohérence

Les créations de compte avec profil, de profil étudiant et d'offre utilisent des transactions SQL. Elles permettent d'annuler ensemble les opérations relationnelles d'un même ensemble lorsque l'une échoue [4]. À l'inverse, la modification d'un profil met à jour l'utilisateur, le profil et éventuellement la référence du CV sans transaction globale. Un échec intermédiaire peut donc laisser une identité modifiée avec des informations de profil plus anciennes. L'ensemble de ces changements représente une unité de travail métier et devrait être traité comme telle.

Une transaction SQL ne couvre pas le déplacement d'un fichier ni un appel HTTP. Le parcours de dépôt peut déplacer le document avant l'enregistrement final du profil. Si cette écriture échoue, un fichier peut subsister sans référence exploitable. La cible proposée consiste à utiliser un emplacement temporaire, enregistrer un état de traitement et prévoir une opération compensatoire de suppression ou de reprise. Elle évite de maintenir une transaction relationnelle ouverte durant une extraction potentiellement lente.

Les vérifications préalables d'existence n'éliminent pas toutes les courses. Deux candidatures simultanées peuvent toutes deux constater l'absence du couple avant l'insertion ; la contrainte unique décide alors laquelle est conservée. L'application doit reconnaître le conflit et retourner une confirmation cohérente. De même, la création concurrente d'un premier score peut produire deux lignes. Enfin, la présélection automatique se fonde sur un état lu avant l'écriture ; une décision humaine concurrente peut être écrasée si l'actualisation ne filtre que l'identifiant. Un contrôle conditionnel sur l'état ou une version de ligne protège cette transition.

Le stockage JSON ajoute un autre problème : la lecture, la modification en mémoire et l'écriture complète du fichier ne sont pas protégées par un verrou commun. Deux requêtes peuvent perdre leurs modifications respectives. Une lecture invalide provoque en outre le remplacement par des données de démonstration. Ce mécanisme ne peut donc pas être assimilé à une stratégie de haute disponibilité. Le passage automatique

d'un support à l'autre fragmente les données, puisque les écritures du fichier ne sont pas réintégrées au retour de MySQL.

Le cycle de lecture SQL contient également une synchronisation automatique du dataset démo. La méthode appelée `seedIfEmpty` n'exige pas réellement que la base soit vide avant cette synchronisation. Une lecture globale peut ainsi réintroduire des enregistrements de démonstration et multiplier les vérifications SQL. La cible doit réserver le peuplement à des commandes explicites, séparées du parcours utilisateur, et rendre le choix du mode de stockage stable pour un environnement donné.

3.6 Architecture cible proposée

La première évolution consiste à extraire l'orchestration des routes vers des contrôleurs et des services applicatifs. Un service de profil coordonnerait validation métier, document et persistance ; un service de candidature porterait les règles d'ouverture d'offre et d'unicité ; un service de décision contrôlerait les transitions d'état. Des politiques d'autorisation centraliseraient les règles de propriété. Cette organisation vise une application Laravel plus lisible, sans imposer un découpage en nouveaux services réseau.

Le stockage de production reposerait sur MySQL comme référence unique. Le mode démonstration serait choisi explicitement et isolé des comptes utilisés réellement. Les dépôts de données disposeraient d'un contrat stable et de méthodes ciblées plutôt que d'un chargement systématique de toutes les entités. Les listes seraient filtrées et paginées en base. Les derniers documents et résultats seraient récupérés en groupe pour éviter une requête supplémentaire par profil ou candidature.

Pour le traitement IA, la cible proposée associe un calcul à une version de profil, une version d'offre et une version d'algorithme. Un changement pertinent rendrait le résultat antérieur obsolète et déclencherait un nouveau traitement. Une file de travaux pourrait exécuter l'extraction et les comparaisons, tandis que l'interface présenterait des états en attente, disponible ou en échec. Ces états et cette file n'existent pas dans le MVP examiné ; leur intérêt devra être confirmé par des mesures de charge et des besoins de réactivité.

Enfin, le suivi métier devrait conserver un historique séparé des décisions, avec leur auteur et leur origine humaine ou automatique. Les traces techniques seraient limitées aux informations nécessaires au diagnostic, sans reproduire inutilement le texte des CV. Des tests de contrat vérifieraient les réponses Python, des tests d'intégration les transactions et autorisations, et des scénarios concurrents l'unicité et la préservation des décisions. Cette cible prépare une consolidation progressive : elle traite les garanties fondamentales avant d'accroître la complexité du moteur de rapproche-

ment.

CHAPTER 4

Réalisation et intégration de SmartRecruit

4.1 Du besoin fonctionnel au prototype exécutable

La réalisation de SmartRecruit traduit le besoin de rapprochement entre étudiants et entreprises en une application Web organisée autour des profils, des offres et des candidatures. Le travail s'inscrit dans le stage de Rahma Amiri chez Best Solutions, du 2 mars au 2 juillet 2026, sous l'encadrement professionnel de Hammami Housseem. Le présent chapitre décrit les mécanismes effectivement présents dans la version du code disponible pour la rédaction. Il distingue leur fonctionnement observable des améliorations envisagées. Cette distinction évite d'attribuer au prototype des capacités qui relèvent encore d'une évolution, notamment en matière d'authentification complète, d'exploitation en production et d'évaluation de la pertinence.

Le produit possède une interface rendue par Laravel, une persistance relationnelle accessible par le constructeur de requêtes du framework et un service Python chargé de traitements textuels. Un moteur PHP permet également de produire un classement lorsque le service externe ne répond pas. Cette organisation rend possible une démonstration progressive : les parcours métier restent compréhensibles sans déployer immédiatement tous les composants. Elle implique toutefois plusieurs modes de fonctionnement, dont les résultats et les garanties doivent être explicités. Une démonstration avec stockage JSON, par exemple, ne constitue pas une validation du comportement transactionnel de MySQL.

4.2 Organisation du code et responsabilités

4.2.1 Une application Laravel à organisation compacte

Le manifeste Composer déclare PHP à partir de la branche 8.2 et Laravel dans la branche majeure 12. L'espace de noms applicatif SmartRecruit est associé au répertoire `app/` par un chargement PSR-4. Le point d'entrée HTTP se trouve actuellement dans `index.php`, à la racine du projet. Il charge les dépendances, initialise l'application définie dans `bootstrap/app.php`, puis transmet la requête capturée au framework. Le

bootstrap enregistre les routes Web, les commandes de console, une route de santé et le middleware de contrôle des rôles.

La structure utilise des routes définies par des fonctions anonymes dans `routes/web.php`. Ce fichier rassemble aussi des fonctions auxiliaires pour sélectionner le stockage, récupérer l'utilisateur courant, composer les textes et ordonner les résultats. Il n'existe donc pas, dans cette version, une séparation complète entre contrôleurs métier, services applicatifs et politiques d'autorisation. Il serait inexact de présenter une architecture à contrôleurs et modèles Eloquent qui ne correspondrait pas aux fichiers livrés. Les conventions Laravel restent mobilisées pour le routage, les réponses, les vues et la validation [6, 7].

Table 4.1: Répartition des responsabilités dans le code disponible

Composant	Responsabilité observée
<code>routes/web.php</code>	Orchestration des parcours, validation HTTP, autorisations liées aux objets et préparation des vues.
<code>EnsureRole</code>	Contrôle du rôle stocké en session avant l'exécution des routes protégées.
<code>SqlStore</code>	Requêtes relationnelles, création des entités, transformation des enregistrements et mémorisation des scores.
<code>DemoStore</code>	Lecture et écriture d'un jeu de données JSON destiné au fonctionnement de démonstration.
<code>CvParser</code>	Coordination de l'extraction du CV et production de métadonnées structurées.
<code>SemanticMatcher</code>	Extraction de compétences, rapprochement lexical et calcul du score local PHP.
<code>AIClient</code>	Appels HTTP vers les fonctions de santé, de matching et d'extraction Python.
<code>resources/views/</code>	Templates Blade des formulaires, tableaux de bord et écrans de consultation.

Cette répartition apporte une lisibilité suffisante pour suivre une candidature de bout en bout. Les deux magasins de données exposent des opérations comparables et retournent des tableaux manipulables par les vues. En revanche, leur interchangeabilité repose sur une convention de méthodes, et non sur une interface explicitement déclarée. La progression naturelle consiste à formaliser ce contrat, puis à déplacer les cas d'utilisation vers des services testables indépendamment du routage. Une telle évolution conserverait les parcours existants tout en réduisant la concentration de responsabilités.

4.2.2 Circulation d'une requête

Une requête traverse d'abord les mécanismes Web de Laravel, notamment la gestion de session et la protection des formulaires contre les requêtes intersites. Le middleware `EnsureRole`, lorsqu'il est associé à la route, détermine ensuite si le rôle courant est accepté. La fonction de route valide les champs, vérifie éventuellement la propriété de l'objet, appelle le stockage et prépare une réponse HTML ou JSON. Les redirections transportent des messages temporaires de succès, d'erreur ou d'avertissement. Cette chaîne explique pourquoi masquer un bouton dans l'interface ne suffit pas : l'autorisation doit également être contrôlée avant l'opération serveur.

4.3 Authentification et maîtrise des accès

4.3.1 Connexion et création de compte actuelles

La version étudiée possède une connexion par adresse électronique et mot de passe. La route de connexion recherche le compte dans le stockage SQL, compare le secret saisi au hash enregistré avec `password_verify`, puis place les informations utiles de l'utilisateur en session en excluant le hash. L'inscription publique accepte uniquement les rôles étudiant et recruteur. Elle contrôle notamment le format et l'unicité de l'adresse électronique, la confirmation du mot de passe et une longueur minimale de huit caractères. Le hash d'un nouveau compte est produit avec `bcrypt`.

L'inscription crée, dans une transaction, le compte et le profil correspondant. Le recruteur fournit une raison sociale ; l'étudiant renseigne un titre de profil. La session devient active après cette création. Les routes de connexion et d'inscription comportent une limitation de dix requêtes par minute. Cette description corrige un décalage documentaire : certains documents du dépôt évoquent encore une connexion de démonstration sans mot de passe. Cette ancienne description ne représente plus le formulaire ni le traitement actuellement présents.

L'authentification demeure spécifique à l'application : les informations sont conservées sous une clé de session propre à SmartRecruit, sans mise en place complète des parcours habituels de récupération du mot de passe, de vérification de l'adresse électronique ou de révocation des sessions. La présence d'un hash ne suffit donc pas à qualifier l'ensemble de solution prête pour la production. La régénération de l'identifiant de session après connexion et l'invalidation complète à la déconnexion figurent parmi les corrections proposées au chapitre 6, conformément aux principes de gestion de session [14].

4.3.2 Rôles et contrôle de propriété

Le middleware distingue l'invité, l'étudiant, le recruteur et l'administrateur. Un invité qui demande une ressource protégée est redirigé vers la connexion ; un utilisateur connecté dont le rôle est incompatible reçoit une interdiction d'accès. Les contrôles portant sur une ressource sont ensuite réalisés dans les routes. L'étudiant ne peut consulter ou modifier que le profil associé à son compte. Le recruteur ne peut consulter, modifier ou demander le classement que pour ses propres offres. Une modification de statut de candidature exige également que l'offre concernée lui appartienne.

Les recruteurs ont accès à la liste des profils et au contenu des CV présentés par l'application. Il s'agit d'un fonctionnement de CVthèque, plus large que la seule consultation des personnes ayant déjà candidaté. Ce choix doit être expliqué aux futurs utilisateurs et confirmé dans les règles métier d'un déploiement réel. Le code réserve les modifications de profils aux étudiants concernés et aux administrateurs. La séparation entre consultation et modification est ainsi déjà matérialisée, même si une politique détaillée de visibilité des données reste à formaliser.

4.4 Parcours d'interface

4.4.1 Navigation et tableaux de bord

L'interface repose sur un layout Blade partagé. La navigation dépend du rôle : les accès aux offres, aux étudiants, au profil personnel et à l'administration apparaissent selon les droits. Les templates utilisent l'échappement de Blade pour restituer les contenus saisis. Les formulaires de mutation comportent un jeton CSRF. Ces mécanismes sont particulièrement utiles pour les descriptions libres et les textes de CV, qui doivent rester des données affichées et non devenir des instructions exécutées par le navigateur.

Le tableau de bord invité présente le principe du matching et quelques compteurs. Celui de l'étudiant regroupe son profil et ses candidatures. Le recruteur consulte ses offres, le nombre de candidatures associées et les meilleurs rapprochements calculés. Des filtres distinguent les états des offres et permettent de retenir celles qui ont reçu des candidatures. L'administrateur dispose d'une vision transversale des offres et des profils, ainsi que d'une console listant les comptes. Ces variantes répondent à des objectifs différents sans multiplier les applications clientes.

La description de ces écrans provient des templates et des traitements de routes. Aucune capture artificielle n'est utilisée pour simuler une campagne de tests visuels ou une observation d'utilisateurs. Une validation ergonomique future devra examiner les

écrans réellement rendus, sur plusieurs largeurs de fenêtre, et vérifier la compréhension des libellés. Le mémoire peut ainsi rendre compte de la réalisation de l'interface sans transformer son existence dans le code en preuve de son utilisabilité.

4.4.2 Profil étudiant et dépôt du CV

L'étudiant inscrit dispose d'un profil qu'il peut compléter avec une localisation, une formation, des années d'expérience, des compétences et des liens professionnels. L'administrateur peut également créer un profil. Le formulaire accepte du texte de CV et un fichier PDF, TXT ou Markdown, dans la limite de 4 Mo. Les champs textuels sont bornés et les liens sont validés comme des URL. La mise à jour associe les compétences saisies manuellement aux compétences extraites lorsqu'un nouveau fichier est fourni.

La page de profil rend visibles le titre professionnel, la formation, les compétences, le nom du document et le texte normalisé. Elle présente aussi les adresses électroniques et les numéros de téléphone détectés par le parseur. Cette restitution joue un rôle de contrôle : l'utilisateur peut repérer une extraction incomplète et corriger les informations. Les métadonnées ne doivent cependant pas être interprétées comme des informations vérifiées. Elles sont issues de règles de reconnaissance et peuvent contenir des erreurs, particulièrement dans les documents à mise en page complexe.

4.4.3 Offres et candidatures

Un recruteur crée une offre avec un intitulé, une entreprise, une description et des compétences demandées. L'offre est publiée par défaut ; le formulaire de modification permet ensuite les états brouillon, publiée et fermée. La liste des étudiants ne présente que les offres publiées, ordonnées selon leur adéquation lorsque le profil est disponible. Le recruteur retrouve ses propres offres, avec des filtres et un classement fondé sur les rapprochements calculés. L'implémentation couvre donc la création, la lecture et la mise à jour ; elle ne fournit pas un CRUD complet comprenant une suppression générique de chaque entité.

La candidature en un clic réutilise le profil déjà enregistré. Elle ne dépose pas un CV distinct pour chaque offre. La page de détail indique ensuite le statut de la candidature de l'étudiant. Du côté recruteur, le classement inclut tous les profils comparés, y compris ceux qui n'ont pas encore candidaté. Les actions de décision apparaissent pour les lignes associées à une candidature. Cette différence entre profil recommandé et candidature reçue est essentielle pour comprendre les nombres affichés et éviter de confondre recherche de talents et traitement d'un dossier déposé.

Les statuts techniques sont `submitted`, `shortlisted`, `interview` et `rejected`.

Le bouton présenté comme une acceptation écrit actuellement le statut `interview`. Il exprime donc une progression dans le processus de sélection, et ne matérialise ni une embauche ni la signature d'une convention de stage. Une évolution des libellés devra rendre cette sémantique plus explicite. De même, le contrôle du caractère publié de l'offre doit être appliqué lors du POST de candidature, pas uniquement lors de l'affichage de sa page.

4.5 Persistance et cohérence métier

Le stockage relationnel sépare comptes, profils étudiants, profils recruteurs, documents de CV, offres, candidatures et scores de rapprochement. Des identifiants publics servent à désigner les objets dans les URL, tandis que les relations internes utilisent les identifiants de base. Les compétences, liens et métadonnées sont sérialisés dans des champs JSON. Le constructeur de requêtes Laravel centralise les accès SQL et les jointures nécessaires aux objets retournés aux vues [4]. Cette organisation évite de disperser des requêtes SQL textuelles dans les templates.

Certaines créations combinant plusieurs enregistrements sont placées dans une transaction. C'est notamment le cas de l'inscription et de la création d'un profil étudiant par le magasin SQL. La mise à jour d'un profil procède, en revanche, par modifications successives du compte, du profil et éventuellement du document. Le déplacement physique du fichier précède aussi l'écriture métier. Ces opérations ne constituent pas une transaction atomique unique : une erreur intermédiaire peut laisser un document orphelin ou un état partiellement actualisé. Le chapitre suivant précise les essais à réaliser dans une base isolée pour caractériser ces situations.

Le système sélectionne automatiquement `SqlStore` lorsque la connexion et plusieurs tables sont disponibles ; sinon il utilise `DemoStore`. Ce repli est utile pour la présentation, mais il s'agit d'un changement de magasin, sans synchronisation automatique des écritures entre JSON et MySQL. Les formulaires de connexion et d'inscription exigent d'ailleurs SQL. Il faut donc éviter de décrire le mode JSON comme une continuité de service équivalente. Pour une exploitation réelle, un mode explicitement configuré et un état d'indisponibilité visible seraient préférables à un basculement implicite.

La lecture globale des données SQL déclenche également la synchronisation du jeu de démonstration. Les comptes et objets de démonstration absents peuvent ainsi être recréés lors de la consultation. Cette facilité doit être retirée du chemin normal des requêtes avant une mise en production et remplacée par une commande d'amorçage volontaire. Les données de présentation, les données de tests et les données réelles doivent avoir des cycles de vie séparés. Le prototype rend le scénario reproductible

; la version exploitable devra garantir que cette reproductibilité ne modifie pas silencieusement les données métier.

4.6 Extraction des CV et intégration du moteur

4.6.1 Chaîne d'extraction

Le composant CvParser construit un nom de fichier simplifié, puis choisit la méthode d'extraction selon l'extension. Pour un PDF, il demande d'abord au service Python le texte du document. Le client envoie le fichier dans une requête multipart avec un délai maximal configuré de six secondes. Il s'agit d'une limite de configuration, et non d'une latence mesurée. Si le service répond correctement et fournit un texte non vide, ce texte est utilisé. Sinon, le parseur applique son traitement local.

Le service Python essaie l'extraction avec `pdfminer.six`, puis un autre lecteur s'il est disponible, avant un dernier décodage des octets. Le premier mécanisme est adapté à la lecture des objets et flux d'un PDF textuel [17]. Le repli local PHP peut lire directement les fichiers textuels ; pour les PDF, son nettoyage par expressions régulières ne remplace pas une analyse de structure. Aucun moteur OCR n'est mis en œuvre. Un CV scanné peut donc rester inexploitable malgré un fichier correctement reçu.

Le texte extrait est combiné au texte manuel. Le parseur recherche ensuite des compétences, une formation, une expérience et des coordonnées. Si le fichier produit moins de quarante caractères et qu'aucun texte manuel ne compense cette faiblesse, un avertissement invite à compléter le profil. Ce seuil est un indicateur pratique, pas une mesure de validité du document : une chaîne suffisamment longue peut être incohérente. La validation du fichier, la qualité de l'extraction et la justesse des métadonnées doivent être testées séparément [15].

4.6.2 Calcul local et appel distant

Le rapprochement commence par un calcul PHP. Si la réponse de santé Python annonce un service disponible, l'application lui transmet le texte de l'offre, le texte du profil et les listes de compétences. Les champs du résultat distant remplacent alors plusieurs valeurs locales, notamment le score et les couvertures. La source du calcul est conservée dans la réponse. Les requêtes JSON utilisent un délai configuré de 0,4 seconde. Les calculs de classement sont exécutés de manière synchrone dans des boucles sur les profils et les offres.

Cette stratégie permet un résultat local lorsque le service externe est inaccessible. Toutefois, les deux moteurs ne sont pas exactement équivalents : leurs dictionnaires,

la canonicalisation des listes et les pondérations diffèrent. Le moteur PHP inclut notamment un bonus d'expérience absent du calcul Python. Une indisponibilité peut donc modifier l'ordre des recommandations, au-delà du changement technique de fournisseur. L'intégration gagnerait à définir un contrat de score versionné et une configuration de compétences commune. Le protocole d'évaluation doit conserver la source et la version de chaque résultat.

Lorsqu'une candidature existe, `SqlStore` peut mémoriser le score et faire passer automatiquement une candidature reçue à l'état présélectionné à partir d'un seuil de 70. Cette écriture peut survenir pendant une consultation qui recalcule un classement. Elle doit être distinguée de la décision humaine exprimée par les boutons. Une séparation future entre calcul, enregistrement du score et changement de statut rendrait le cycle de décision plus prévisible, testable et traçable.

4.7 Configurations d'exécution et déploiement

La figure 4.1 représente les services et les ports déclarés dans la composition de démonstration.

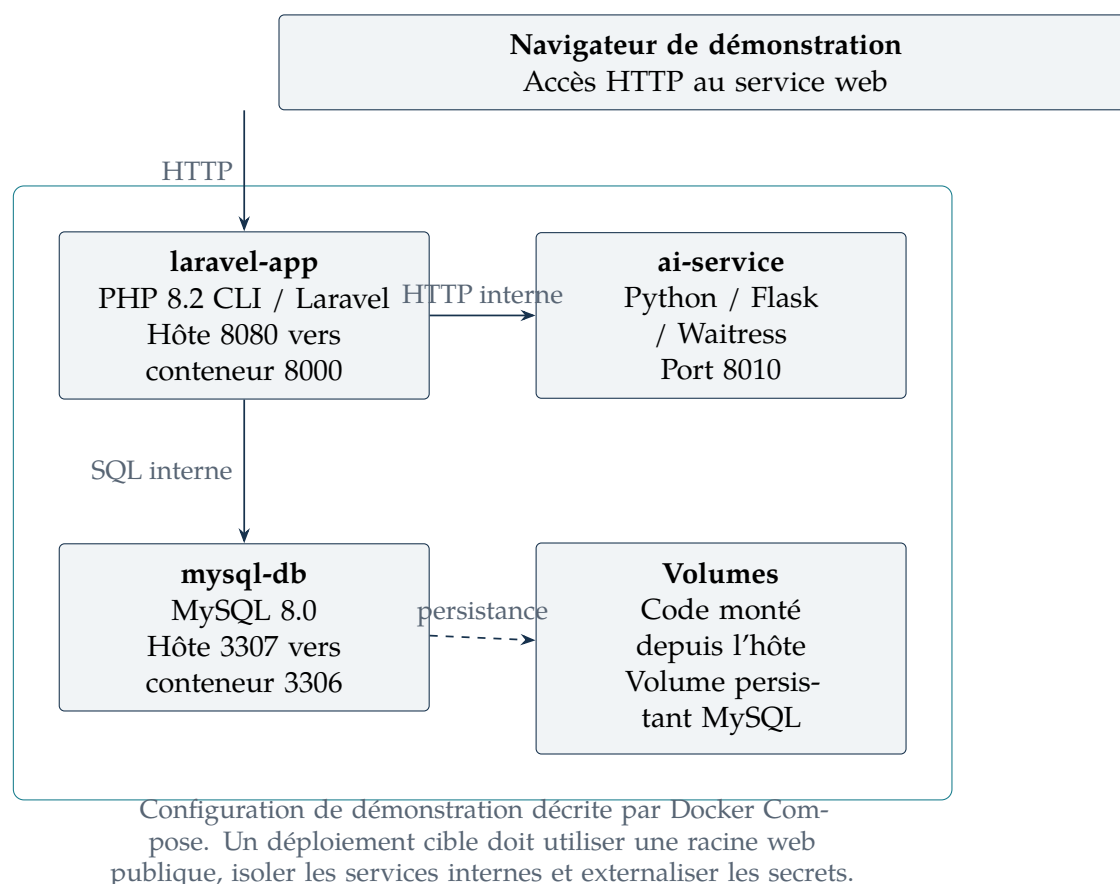


Figure 4.1: Configuration de démonstration décrite par Docker Compose

4.7.1 Exécution locale, Apache et conteneurs

Le dépôt propose un démarrage local avec le serveur intégré de PHP et un script de routage. Ce mode facilite la démonstration sur un poste de développement. Le fichier `server.php` laisse toutefois le serveur servir directement les fichiers existants sous la racine du projet. La distinction entre ressources publiques et fichiers internes n'est donc pas assurée par ce mode. Le serveur intégré est destiné au développement ; il ne doit pas devenir, par simple réutilisation de la commande, le serveur d'une exploitation ouverte [21].

Une configuration Apache est également présente dans `.htaccess`. Elle contient des interdictions visant des fichiers de configuration et plusieurs répertoires internes, puis réécrit les demandes vers le point d'entrée. Ces règles sont spécifiques à Apache et dépendent de son paramétrage ; le serveur intégré PHP ne les applique pas. Leur présence dans le dépôt ne démontre donc pas que tous les modes d'exécution protègent les mêmes chemins. Une racine publique dédiée, conforme à l'organisation recommandée pour le déploiement Laravel, simplifierait cette frontière [5].

Docker Compose décrit trois services : application Laravel, service Python et base MySQL. Le service Web construit une image PHP, installe les dépendances et démarre le même serveur intégré. Le service Python installe ses bibliothèques et expose les routes d'analyse ; lorsqu'il est disponible, Waitress sert l'application Flask. Un volume conserve les données MySQL et les services partagent un réseau Compose. Cette description matérialise l'architecture distribuée, mais les déclarations de dépendances de démarrage ne remplacent pas une vérification de disponibilité opérationnelle [3, 16].

Les fichiers livrés décrivent un environnement de démonstration avec ports publiés et paramètres de développement. Aucune construction d'image ni exécution de cette composition n'est revendiquée dans l'évaluation présentée. La préparation d'un déploiement réel nécessitera de vérifier la compatibilité des dépendances, de maîtriser les secrets, de restreindre les services accessibles et de définir les vérifications de santé. Ces actions sont des travaux proposés, pas des propriétés déjà validées de l'installation.

4.7.2 Frontière entre démonstration et production

Le prototype réunit les principales interactions attendues : création d'un compte, enrichissement d'un profil, publication d'une offre, recommandation, candidature et suivi. Cette continuité fonctionnelle constitue son apport concret. La production ajoute des exigences différentes : conservation fiable des documents, autorisations vérifiables, reprise après incident, politique de sauvegarde, diagnostic exploitable et contrôle des versions du moteur. Il convient de traiter ces exigences comme des

critères d'acceptation explicites, plutôt que comme des conséquences automatiques du choix de Laravel ou de Docker.

La démarche retenue consiste ainsi à préserver les mécanismes utiles, à isoler les facilités de démonstration et à mesurer les limites avant d'ajouter des fonctionnalités. Le chapitre 6 relie les vérifications réellement effectuées aux scénarios encore à exécuter. Il fournit une base de progression où chaque correction proposée peut être associée à une preuve attendue, sans confondre réalisation du prototype et qualification complète d'un système de recrutement.

CHAPTER 5

Conception et fonctionnement du moteur de matching

5.1 Organisation de la chaîne de traitement

Le moteur transforme une offre et un profil étudiant en un ensemble d'indicateurs, puis en un score entier compris entre 0 et 100. Son fonctionnement repose sur quatre composants principaux : `app/Support/CvParser.php` prépare le contenu d'un CV ; `app/Support/SemanticMatcher.php` réalise le calcul local ; `app/Support/AiClient.php` communique avec le microservice ; `ai-service/app.py` expose l'extraction PDF et une seconde implémentation du score. L'orchestration des appels se trouve dans `routes/web.php`.

La figure 5.1 présente cette chaîne. Le fichier constitue une source possible du profil, auquel s'ajoutent les champs saisis. Le rapprochement exploite ensuite deux représentations complémentaires : des ensembles de compétences et des vecteurs lexicaux. Les catégories sont déduites des compétences reconnues. Aucun modèle neuronal n'est chargé dans ce parcours et aucun apprentissage de coefficients n'est réalisé à l'exécution.

Le texte de l'offre concatène son titre, sa description et les compétences requises. Celui du candidat concatène son intitulé de profil, sa formation, le texte de son CV et ses compétences. Cette composition est importante : une compétence présente à la fois dans le CV et dans un champ structuré peut apparaître plusieurs fois dans le texte. Les ensembles de compétences sont dédoublés, mais les fréquences lexicales conservent ces répétitions. Les champs structurés influencent donc les deux branches du calcul.

5.2 Extraction et préparation du CV

5.2.1 Sources textuelles et extraction PDF

La méthode `CvParser::parse` accepte un fichier optionnel et un texte manuel. Lorsqu'un fichier est fourni, elle tente d'en extraire le contenu, puis concatène le

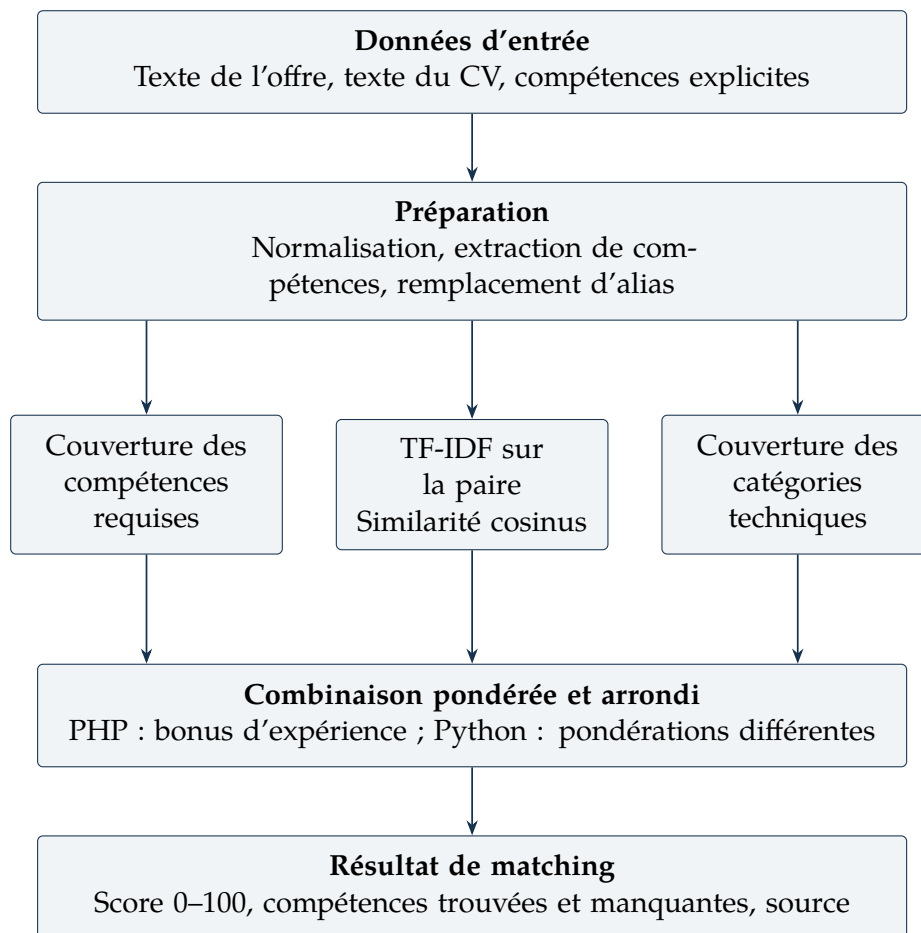


Figure 5.1: Chaîne de préparation des données et de calcul du matching dans SmartRecruit

résultat avec le texte manuel. Le texte saisi complète ainsi le contenu extrait ; il ne le remplace pas automatiquement. La méthode renvoie notamment le texte final, les compétences, une estimation de l'expérience, une formation détectée et des métadonnées de contact.

Pour un fichier portant l'extension PDF, le parseur appelle d'abord `AiClient::parseCv` lorsque le client est disponible. L'appel transmet le fichier en multipart au point d'accès `/parse-cv`, avec un délai maximal configuré de six secondes. Côté Python, un fichier temporaire est créé, alimenté puis supprimé dans un bloc `finally`. Le mécanisme d'accès aux fichiers reçus correspond au modèle décrit par Flask pour les téléversements [16].

L'extraction Python tente d'abord `pdfminer.high_level.extract_text`. En cas d'exception, elle essaie l'interface PyPDF2 présente dans le code. Si ces tentatives échouent, elle lit les octets et les décode en Latin-1. Ce dernier comportement est une limite : obtenir une chaîne de caractères à partir d'un PDF ne signifie pas avoir extrait son contenu rédactionnel. De plus, lorsque `pdfminer` retourne une chaîne vide sans exception, la fonction renvoie directement cette chaîne et ne poursuit pas la tentative PyPDF2.

En l'absence de résultat Python non vide, PHP utilise une extraction locale. Les fichiers TXT, Markdown et CSV sont lus comme du texte brut. Pour un PDF, une expression régulière retire certains délimiteurs de parenthèses, puis un filtrage conserve des lettres, chiffres et caractères particuliers. Cette procédure ne décompresse pas les flux PDF et ne reconstruit pas leur structure. Aucune étape de reconnaissance optique de caractères n'est implémentée pour les pages numérisées sous forme d'images.

Le champ `extraction_unreliable` devient vrai lorsqu'un fichier a été fourni, que le texte extrait contient moins de 40 caractères et qu'aucun texte manuel ne compense cette situation. Cette garde rend visible un échec simple, mais sa portée reste limitée. Un contenu technique de plus de 40 caractères peut franchir le seuil sans être un CV exploitable. La correction proposée consiste à retourner un statut d'extraction distinct du texte, plutôt qu'à interpréter uniquement sa longueur ; elle ne fait pas partie du code étudié.

5.2.2 Compétences, formation et expérience

Les compétences et les coordonnées sont recherchées dans le texte final. Les adresses électroniques et les numéros de téléphone sont extraits par expressions régulières, sans validation de leur caractère opérationnel. La formation est déterminée par la première correspondance dans une liste ordonnée : doctorat, master, mastère, ingénieur, licence, bachelor et BTS. Ce mécanisme ne reconstitue pas un parcours académique. Il peut

retenir un diplôme mentionné comme objectif ou dans une autre partie du document.

L'expérience est obtenue en recherchant un entier suivi d'une expression telle que *ans*, *années* ou *year(s)*. La présence du mot *expérience* est facultative dans l'expression régulière. La méthode retient le maximum des nombres détectés et retourne zéro en l'absence de correspondance. Elle ne calcule pas la durée d'intervalles datés, ne cumule pas des stages et ne gère pas leurs chevauchements.

Cette règle explique qu'une mention d'âge ou de durée d'études puisse devenir une expérience professionnelle. L'incidence est directe puisque cette estimation peut alimenter le profil et le bonus du moteur PHP. Il conviendrait de conserver la provenance de la valeur, de limiter l'inférence aux formulations explicitement professionnelles et de permettre sa vérification. Les sondes ciblées du chapitre 6 illustrent ce problème ; elles ne mesurent pas une précision globale d'extraction sur un corpus de CV.

5.3 Normalisation, alias et catégories

5.3.1 Traitement lexical effectivement implémenté

Le moteur PHP transforme le texte en minuscules avec `mb_strtolower`, puis tente une translittération vers ASCII avec `iconv`. Il remplace ensuite par des espaces les caractères hors de l'ensemble formé des lettres ASCII, chiffres, signes plus, dièse et point. Les espaces successifs sont réduits. Le moteur Python applique une mise en minuscules et un filtrage comparable, mais ne réalise pas la translittération préalable : une lettre accentuée est éliminée par le filtre ASCII. La normalisation du français ne peut donc pas être considérée comme identique dans les deux langages.

L'extraction des compétences recherche chaque alias normalisé avec un espace avant et après. Cette précaution évite certaines correspondances au milieu d'un mot. Cependant, le point est conservé par la normalisation : *Python.* ne correspond pas à l'alias *python* entouré d'espaces. La conservation de caractères utiles aux noms techniques nécessite ainsi un traitement plus précis de la ponctuation terminale.

La tokenisation constitue une opération différente. Après normalisation, les alias sont remplacés successivement par leur nom canonique ; les espaces internes de ce nom deviennent des underscores. Le texte est découpé sur les espaces, puis les tokens de longueur inférieure ou égale à deux et les mots vides français ou anglais sont supprimés. Les substitutions n'utilisent pas les limites de mots de l'extracteur. Leur ordre peut modifier un alias avant qu'un alias plus long soit examiné : *Spring Boot* devient ainsi *java boot*, tandis que *React.js* peut produire *react.javascript*.

5.3.2 Rôle du dictionnaire et de la taxonomie

Le dictionnaire PHP défini dans `config/smart_recruit.php` contient 21 compétences canoniques. Celui de Python en contient 16. Les entrées Angular, NoSQL, cloud, mobile et sécurité sont présentes uniquement côté PHP. Certaines correspondances expriment une famille technique : Laravel et Symfony sont associés à PHP ; Spring et Spring Boot à Java ; pandas et NumPy à l'analyse de données. Ce sont des règles de rapprochement retenues dans le prototype, et non des équivalences exhaustives entre niveaux de maîtrise.

La taxonomie associe ensuite chaque compétence connue à une catégorie : Java, PHP et Python au développement backend ; React et Vue au frontend ; SQL aux données ; DevOps à l'infrastructure. Les catégories sont dédoublées. Posséder plusieurs compétences backend n'apporte donc qu'une catégorie à ce niveau. Cette réduction permet de valoriser une proximité de domaine même sans identité de technologie, mais elle peut aussi rapprocher des connaissances professionnelles très différentes.

Le traitement des compétences explicitement saisies diverge également. PHP les canonise : pour chaque chaîne, il retient la première compétence détectée ou, à défaut, sa forme normalisée. Python conserve les valeurs reçues et les réunit aux compétences extraites du texte, sans canonisation préalable. Une valeur explicite *Laravel* peut ainsi coexister avec la compétence extraite *php*. Cette duplication conceptuelle modifie le dénominateur de la couverture et peut créer un manque artificiel.

5.4 Définition des composantes mathématiques

5.4.1 Couverture des compétences et des catégories

Soient R l'ensemble des compétences considérées comme requises et P l'ensemble des compétences du profil, après les traitements propres au moteur utilisé. Les compétences communes sont $M = R \cap P$ et les manques sont $D = R \setminus P$. La couverture des compétences est définie par :

$$C(R, P) = \begin{cases} \frac{|R \cap P|}{|R|}, & \text{si } |R| > 0, \\ 0, & \text{sinon.} \end{cases} \quad (5.1)$$

Cette mesure est orientée par le besoin : des compétences supplémentaires du candidat n'augmentent pas directement C , mais des compétences requises supplémentaires peuvent le diminuer. Toutes les compétences ont le même poids. Le code ne distingue pas une obligation d'une préférence lorsqu'il extrait les compétences de la

description de l'offre. La mention d'une technologie dans une phrase contextuelle peut donc enrichir R comme une exigence explicite.

En notant $\Gamma(A)$ l'ensemble des catégories connues associées aux compétences de A , la couverture sémantique du code est :

$$K(R, P) = \begin{cases} \frac{|\Gamma(R) \cap \Gamma(P)|}{|\Gamma(R)|}, & \text{si } |\Gamma(R)| > 0, \\ 0, & \text{sinon.} \end{cases} \quad (5.2)$$

Les compétences absentes de la taxonomie ne participent pas à Γ . Si l'offre n'exige que Java et que le profil ne mentionne que PHP, les catégories backend coïncident : $K = 1$ tandis que $C = 0$. Ce résultat exprime le choix de granularité de la taxonomie. Il ne prouve pas que le candidat satisfait l'exigence Java.

5.4.2 TF-IDF calculé sur une paire de documents

Soient d_o et d_c les listes de tokens de l'offre et du candidat. Le corpus du calcul est toujours la paire $\mathcal{D} = (d_o, d_c)$, avec deux documents comptés séparément même si leurs textes sont identiques. Pour un terme t , $n(t, d)$ désigne son nombre d'occurrences et $|d|$ le nombre de tokens du document. Les deux implémentations utilisent :

$$\text{tf}(t, d) = \frac{n(t, d)}{\max(|d|, 1)}, \quad (5.3)$$

$$\text{df}(t, \mathcal{D}) = \sum_{d \in \mathcal{D}} \mathbf{1}_{\{t \in d\}}, \quad (5.4)$$

$$\text{idf}(t, \mathcal{D}) = \ln \left(\frac{1 + 2}{1 + \text{df}(t, \mathcal{D})} \right) + 1, \quad (5.5)$$

$$v_d(t) = \text{tf}(t, d) \text{idf}(t, \mathcal{D}). \quad (5.6)$$

La constante ajoutée évite une pondération nulle pour un terme commun. Un terme présent dans les deux documents possède une IDF de 1 ; un terme présent dans un seul document possède une IDF de $1 + \ln(3/2)$, soit environ 1,405465. Ces deux valeurs épuisent les cas pour les termes effectivement présents. Le moteur n'estime donc pas la rareté d'une technologie dans une population de candidats.

Le cosinus est calculé sur l'union des termes des deux documents :

$$T(d_o, d_c) = \begin{cases} \frac{\sum_t v_{d_o}(t) v_{d_c}(t)}{\sqrt{\sum_t v_{d_o}(t)^2} \sqrt{\sum_t v_{d_c}(t)^2}}, & \text{si les deux normes sont non nulles,} \\ 0, & \text{sinon.} \end{cases} \quad (5.7)$$

La division par la longueur du document dans TF multiplie chaque vecteur par

un facteur uniforme, qui s'annule dans le cosinus lorsque les normes sont non nulles. Elle ne supprime pas l'effet du vocabulaire supplémentaire : de nouveaux termes modifient la direction du vecteur. De même, répéter sélectivement une compétence peut modifier les fréquences relatives. Le calcul conserve donc une sensibilité à la rédaction du profil.

5.5 Formules finales PHP et Python

En notant e la valeur numérique de `experience_years`, avec zéro par défaut, le bonus PHP est défini par :

$$B(e) = 0,05 \min\left(\frac{e}{5}, 1\right). \quad (5.8)$$

Il atteint cinq points à partir de cinq ans pour une entrée non négative. Le code ne borne pas séparément ce bonus vers le bas ; la borne générale du score s'applique ensuite. Avec $\text{clip}(x) = \max(0, \min(100, x))$, les formules exactes sont :

$$S_{\text{PHP}} = \text{clip}(\text{arrondi}_{\text{PHP}}[100(0,55C + 0,30T + 0,10K + B(e))]), \quad (5.9)$$

$$S_{\text{Python}} = \text{clip}(\text{arrondi}_{\text{Python}}[100(0,58C + 0,32T + 0,10K)]). \quad (5.10)$$

Les arrondis désignent les fonctions natives appelées par chaque langage, puis la conversion en entier. Python emploie l'arrondi au pair pour les valeurs exactement à mi-chemin ; PHP utilise par défaut un arrondi des demis en s'éloignant de zéro. Une recherche d'équivalence doit donc couvrir également les valeurs frontières et la représentation flottante, au-delà de l'égalité des coefficients.

Les coefficients sont fixés dans le code. Ils ne résultent pas d'une optimisation sur un corpus annoté. Avec $C = T = K = 1$ et $e = 0$, PHP retourne 95 tandis que Python retourne 100. En l'absence de toute compétence requise reconnue ou déclarée, C vaut zéro ; lorsque la taxonomie requise est vide, K vaut également zéro. Même des textes identiques peuvent alors obtenir un score limité par les seuls coefficients du cosinus et, en PHP, de l'expérience.

PHP ajoute une explication selon des seuils de 80 et 60, puis selon la présence de compétences communes ou manquantes. Ces textes sont des descriptions de classes de score, pas des justifications causales détaillées. Python ne renvoie pas ce champ. Les deux moteurs utilisent pourtant le même identifiant d'algorithme, `tf-idf-cosine+semantic-skill-taxonomy`, ce qui ne suffit pas à identifier précisément les règles exécutées.

5.6 Exemple numérique reproductible

Considérons une offre dont le texte est *java react* et un profil dont le texte est *java sql*. Les autres champs textuels et les listes explicites sont vides ; l'expérience vaut deux ans pour le calcul PHP. Les tokens restent inchangés. Les compétences requises sont $R = \{\text{java}, \text{react}\}$ et celles du profil $P = \{\text{java}, \text{sql}\}$. Ainsi, $M = \{\text{java}\}$, $D = \{\text{react}\}$ et $C = 1/2$.

Les catégories de l'offre sont backend et frontend ; celles du profil sont backend et data. Leur intersection contient une catégorie sur deux, d'où $K = 1/2$. Posons $a = 1 + \ln(3/2)$. Dans l'ordre de dimensions Java, React, SQL, les vecteurs sont :

$$v_o = \left(\frac{1}{2}, \frac{a}{2}, 0\right), \quad v_c = \left(\frac{1}{2}, 0, \frac{a}{2}\right). \quad (5.11)$$

Le produit scalaire vaut $1/4$ et chaque norme vaut $\sqrt{1+a^2}/2$. Il en résulte :

$$T = \frac{1}{1+a^2} \simeq 0,336096927. \quad (5.12)$$

Les contributions sont détaillées dans le tableau 5.1. Les valeurs intermédiaires gardent ici plusieurs décimales pour rendre le calcul vérifiable ; elles ne constituent pas une mesure expérimentale de pertinence.

Table 5.1: Contributions en points pour l'exemple Java, React et SQL

Composante	PHP	Python
Couverture des compétences	27,50	29,00
Similarité textuelle	10,082908	10,755102
Couverture des catégories	5,00	5,00
Expérience de deux ans	2,00	0,00
Total avant arrondi	44,582908	44,755102
Score retourné	45	45

Les deux moteurs donnent donc le même entier dans cet exemple malgré des calculs différents. Ils retournent une similarité textuelle arrondie à 34 et une couverture des catégories de 50. La compétence commune est Java et la compétence manquante est React. Cette égalité ponctuelle montre pourquoi quelques exemples positifs ne suffisent pas à démontrer la parité de deux implémentations.

5.7 Contrat HTTP et mécanisme de repli

Le service expose trois points d'accès. GET/health retourne un statut et l'identifiant de l'algorithme. POST/match reçoit offer_text, cv_text, required_skills et candidate_skills, puis retourne le score, les compétences communes et manquantes, les deux indicateurs et la source. POST/parse-cv reçoit un champ fichier et retourne du texte, des adresses électroniques et des compétences. L'expérience ne figure pas dans la requête de matching Python.

Le contrôle de disponibilité est réalisé avant une série de comparaisons. Pour chaque paire, la fonction sr_match calcule d'abord le résultat PHP. Lorsque le service est déclaré disponible, elle lui demande un résultat et remplace certains champs si la réponse contient un score. Le délai configuré pour les appels JSON de AiClient est de 0,4 seconde. Une absence de réponse exploitable conserve le résultat local. Cette organisation protège la disponibilité fonctionnelle, mais ajoute un calcul local même lorsqu'un score Python sera ensuite retenu.

La validation du contrat reste partielle. Le client JSON accepte une réponse décodée en tableau sans contrôler explicitement le statut HTTP. L'orchestrateur exige la présence du score mais ne vérifie pas l'ensemble du schéma. Le point d'accès Python accepte aussi les données sans validation systématique des types. Une requête contenant un nombre à la place d'un texte peut donc déclencher une exception. Ces observations motivent un schéma d'entrée et de sortie commun et des réponses d'erreur contrôlées, proposés comme améliorations.

Le démarrage Python prévoit plusieurs modes. Lorsque Flask et Waitress sont disponibles, Waitress sert l'application WSGI. Sans Waitress, le serveur intégré de Flask est utilisé. Sans Flask, un serveur de secours fondé sur la bibliothèque standard expose encore la santé et le matching, mais retourne 501 pour l'extraction multipart PDF. Un statut de santé positif ne garantit donc pas à lui seul la disponibilité de toutes les capacités du service.

Les classements sont construits par comparaisons synchrones. Pour n profils et m offres, la recommandation du meilleur rapprochement de chaque étudiant peut effectuer $n \times m$ calculs et autant d'appels Python. Il s'agit d'une propriété des boucles présentes, pas d'une mesure de latence. Un traitement par lots ou une mise en cache exigerait une clé tenant compte des textes, compétences et versions du moteur pour éviter de réutiliser un score devenu obsolète.

5.8 Divergences constatées et trajectoire de correction

Les différences décrites ont des conséquences observables. Avec des textes identiques *angular aws mongodb*, sans listes explicites ni expérience, le moteur PHP reconnaît trois compétences et retourne 95 ; Python n'en reconnaît aucune et retourne 32. Ce cas déterministe, repris au chapitre 6, démontre une divergence de contrat métier. Il ne permet pas de conclure à une supériorité générale du moteur PHP sur un jeu de recrutement.

L'intégration présente une autre incohérence : lors du remplacement du résultat PHP par celui de Python, l'explication PHP est conservée. Elle peut donc décrire une forte compatibilité alors que le score final est faible. La correction devrait produire l'explication à partir des indicateurs finalement retenus, ou inclure une explication vérifiée dans le contrat commun. Elle devrait aussi identifier la version du dictionnaire et de la formule, car la seule source `local-php` ou `python-ai` est insuffisante pour reproduire durablement un résultat.

Une première évolution consisterait à externaliser le dictionnaire, la taxonomie, les coefficients et les règles d'arrondi dans une spécification partagée. Les deux moteurs devraient utiliser la même normalisation des compétences explicites et des alias. Les substitutions devraient respecter les limites de mots et traiter les expressions longues avant leurs fragments. Cette évolution suppose des tests de parité sur des entrées identiques, notamment pour les accents, la ponctuation et les noms techniques composés.

Une seconde évolution concernerait la qualité des données. Le parseur devrait distinguer texte extrait, contenu illisible et saisie manuelle ; les métadonnées inférées devraient conserver leur provenance. Les erreurs du service devraient être observables sans journaliser les CV. Enfin, la cohérence fonctionnelle du repli devrait être testée indépendamment de sa disponibilité réseau : un service de secours est utile seulement si son résultat garde une interprétation maîtrisée.

Ces propositions décrivent un travail restant à réaliser. Elles précèdent logiquement l'ajout d'un modèle appris, car une vectorisation plus élaborée ne corrigerait pas un PDF mal extrait ni des champs incohérents. Une expérimentation future pourrait comparer le moteur stabilisé à des représentations de phrases, mais devrait conserver des entrées, annotations et critères communs. Dans l'état étudié, la contribution est un calcul heuristique explicable dont les formules, conditions de fonctionnement et limites peuvent être examinées et reproduites.

CHAPTER 6

Validation du prototype et perspectives d'amélioration

6.1 Périmètre et statut des résultats

L'évaluation de SmartRecruit poursuit trois objectifs : vérifier que le code disponible constitue un ensemble techniquement cohérent, examiner quelques comportements déterminants et définir les preuves nécessaires avant une utilisation réelle. Elle porte sur le prototype décrit au chapitre 4. Les contrôles présentés ont été réalisés lors de l'analyse du code disponible en septembre 2026. Ils ne sont pas attribués rétrospectivement au stage de Rahma Amiri chez Best Solutions, qui s'est déroulé du 2 mars au 2 juillet 2026. Cette précision chronologique distingue les réalisations du projet des vérifications complémentaires effectuées pour documenter le mémoire.

Les résultats ne constituent ni une qualification de production ni une étude statistique de la qualité du recrutement. Les vérifications locales n'ont pas exécuté de campagne de charge, d'étude d'utilisabilité, de tests avec des recruteurs, de déploiement Docker ou de scénario complet sur la base métier existante. Aucun taux de précision global ni aucune latence de service ne sont donc annoncés. Les chiffres rapportés correspondent exclusivement aux sorties des contrôles identifiés. Les corrections décrites dans ce chapitre sont proposées ; leur présentation ne signifie pas qu'elles ont été appliquées au code.

6.2 Méthodologie de vérification

6.2.1 Trois niveaux de preuve

Le premier niveau consiste à lire le code et les configurations : routes, middleware, formulaires, magasins de données, parseurs et scripts de lancement. Il permet d'établir l'existence d'un contrôle ou d'un chemin d'exécution, mais pas de prouver le succès d'une transaction sur un serveur réel. Le deuxième niveau repose sur des exécutions locales ciblées : analyse syntaxique PHP, inspection des routes, vérification des dépendances et sondes portant sur les fonctions de calcul ou les frontières HTTP. Le troisième niveau rassemble les essais proposés, qui nécessitent un environnement

isolé et un jeu de données préparé.

Ces niveaux sont distingués dans les tableaux et dans le vocabulaire employé. Un défaut observé par lecture est décrit comme une propriété du code ou comme un risque conditionnel. Un résultat exécuté comporte son entrée de référence et sa sortie observable. Un test proposé précise l'oracle attendu, c'est-à-dire la règle permettant de conclure au succès ou à l'échec. Cette discipline empêche de convertir une liste de scénarios souhaitables en bilan de tests prétendument réalisés.

6.2.2 Reproductibilité et protection des données

Les sondes complémentaires sont regroupées dans `evidence/probes.php` et `evidence/probes.py`, avec des résultats sérialisés en JSON dans le dossier d'accompagnement du mémoire. Les entrées sont synthétiques et les fonctions sont sollicitées sans utiliser les CV personnels existants. L'environnement local relevé comprend PHP 8.2.12 en ligne de commande, Composer 2.7.6 et Python 3.12.3. Ces versions décrivent le poste d'analyse ; elles ne prouvent pas la compatibilité de toutes les images ou versions suggérées ailleurs dans le dépôt.

La vérification de la surface HTTP utilise uniquement un manifeste logiciel non sensible. Aucune requête de lecture de secret ou de document personnel n'est nécessaire pour démontrer que le serveur sert un fichier interne. Le serveur local temporaire a été arrêté après l'essai. Les contrôles isolés du proxy utilisent des adresses de documentation et n'envoient pas de tentatives de connexion vers un système externe. Cette approche maintient une preuve utile tout en limitant les effets de l'audit sur les données et les services.

6.3 Résultats des contrôles techniques locaux

Le succès de l'analyse syntaxique élimine une catégorie immédiate d'erreurs, comme une accolade manquante ou une expression PHP invalide. Il ne vérifie ni les droits, ni les types retournés par le service Python, ni la disponibilité des tables. De même, la validation du manifeste et le contrôle des exigences de plateforme constituent un socle reproductible, mais ne garantissent pas que chaque dépendance reste adaptée à un autre environnement. L'avertissement relatif à la licence doit être traité avant une diffusion, sans être présenté comme un échec d'exécution.

L'inventaire des routes a permis de relier chaque parcours décrit aux fonctions correspondantes et d'identifier les contrôles explicitement associés. Les routes protégées comportent des restrictions de rôles ; plusieurs opérations vérifient aussi la propriété des offres ou des profils. Cette lecture fournit un point de départ pour une

Table 6.1: Contrôles exécutés sur la version disponible en septembre 2026

Contrôle	Résultat observé	Portée de la conclusion
Analyse syntaxique PHP	20 fichiers contrôlés, sans erreur de syntaxe	Les fichiers analysés sont interprétables syntaxiquement ; les parcours métier ne sont pas tous exécutés.
Inspection du routage	23 routes recensées	Les points d'entrée et leurs middleware peuvent être examinés ; leur présence ne vaut pas test fonctionnel.
Validation Composer	Manifeste valide ; avertissement sur l'absence de licence	Cohérence du manifeste vérifiée, sans conclusion sur les droits de diffusion du projet.
Exigences de plateforme	Contrôle hors dépendances de développement satisfait	Les exigences déclarées sont satisfaites par l'environnement PHP local utilisé.
Ressource interne HTTP	<code>/composer.json</code> retourne HTTP 200	Le lancement PHP fourni sert directement un fichier interne sous la racine du projet.
Confiance proxy	L'IP déclarée par l'entête devient l'IP client	La confiance globale influe sur l'identité réseau utilisée par le framework.

matrice de tests d'autorisation. Pour chaque ressource, cette matrice devra croiser un invité, son propriétaire, un utilisateur du même rôle mais d'un autre compte et l'administrateur. L'oracle dépendra de la règle métier, pas simplement de l'absence d'une erreur serveur.

6.4 Comportements du moteur de rapprochement

6.4.1 Sondes de cohérence entre PHP et Python

Deux sondes locales mettent en évidence que les moteurs PHP et Python ne produisent pas systématiquement le même score. La première utilise des textes identiques centrés sur Angular, AWS et MongoDB, sans compétences explicites et avec une expérience nulle. Le moteur PHP reconnaît les familles présentes dans sa configuration ; le dictionnaire Python ne dispose pas de la même couverture. La seconde porte sur une représentation explicite de compétences utilisant Laravel et PHP. Les fixtures exactes sont conservées avec les sondes afin de ne pas réduire la comparaison à quelques nombres détachés de leurs entrées.

Ces valeurs sont des scores applicatifs sur une échelle de zéro à cent. Elles ne représentent pas des pourcentages de candidats correctement classés ni une probabilité de réussite professionnelle. Les deux moteurs pondèrent différemment les composantes et le calcul PHP ajoute un bonus d'expérience. Il serait donc abusif de

Table 6.2: Sondes ciblées de comparaison des deux implémentations

Cas synthétique	Score PHP	Score Python	Interprétation
Textes Angular, AWS, MongoDB ; listes explicites vides ; expérience nulle	95	32	Couverture différente des dictionnaires et des familles de compétences.
Compétences exprimées avec Laravel et PHP	95	71	Canonicalisation des listes et règles de calcul non équivalentes.

conclure qu'un moteur est globalement meilleur parce qu'il attribue une valeur plus élevée à ces exemples. Le résultat établi est une absence d'équivalence, susceptible de modifier le classement lorsqu'un appel distant remplace le résultat local.

La correction proposée consiste à externaliser un référentiel versionné de compétences, à appliquer les mêmes règles de canonicalisation aux listes explicites et au texte extrait, puis à décider si les deux moteurs doivent partager une formule ou assumer deux modèles distincts. Dans le premier cas, des tests de contrat devront comparer les composantes sur un corpus de fixtures. Dans le second, l'interface et les données enregistrées devront conserver clairement la version du moteur. Une tolérance numérique ne doit pas masquer une divergence sémantique de dictionnaire.

6.4.2 Expérience et qualité de l'extraction

Une sonde textuelle contenant l'expression « 24 ans » a produit une expérience de 24 années. La règle PHP accepte une quantité suivie d'une unité d'année, tandis que la mention d'expérience est facultative. Une indication d'âge peut ainsi être interprétée comme une expérience professionnelle. L'erreur est particulièrement significative puisque cette variable contribue au bonus local. Une correction doit exiger un contexte professionnel explicite, distinguer les périodes datées des quantités déclarées et autoriser l'absence de résultat lorsque le texte reste ambigu.

Le test de document a fourni au service un contenu synthétique qui n'était pas un PDF valide. La route a néanmoins répondu HTTP 200 avec 173 caractères de texte. Le mécanisme de repli qui décode les octets peut expliquer cette réponse : l'échec des parseurs spécialisés ne devient pas nécessairement une erreur de document. Ce constat concerne la route Python testée directement. Il ne démontre pas que le même contenu franchirait la validation de fichier du formulaire Laravel. Les deux frontières doivent faire l'objet d'essais séparés.

Un traitement renforcé devrait distinguer document reconnu, texte extrait, extrac-

tion insuffisante et échec de format. La quantité de texte ne suffit pas : des octets décodés peuvent former une longue chaîne dépourvue d'information professionnelle. Les tests proposés couvriront PDF textuels, scans, documents tronqués, fichiers textuels renommés et caractères accentués. La chaîne d'extraction doit restituer un diagnostic exploitable, sans confondre l'acceptation d'une requête HTTP avec la validité d'un CV [15, 17].

6.5 Accès, sessions et frontière de déploiement

6.5.1 Exposition observée avec le serveur intégré

Le test HTTP sur `/composer.json` retourne le manifeste en réponse 200 lorsque le projet est lancé avec la commande fournie. Le script de routage accepte tous les fichiers existants sous la racine comme des ressources statiques. Cette preuve suffit à établir que la frontière publique n'est pas correctement définie dans ce mode. L'exposition potentielle de fichiers sensibles découle de cette règle de routage ; leur contenu n'a pas été consulté. Les interdictions Apache présentes dans `.htaccess` ne sont pas exécutées par le serveur intégré PHP.

La correction proposée est de placer uniquement le point d'entrée et les ressources destinées au navigateur dans une racine publique dédiée, puis de faire pointer chaque mode de lancement vers cette racine. Une vérification de non-régression devra demander plusieurs chemins internes et constater un refus ou une absence de ressource, tout en confirmant le chargement normal des ressources publiques. Cette modification traite la cause structurelle et facilite la conformité aux recommandations de déploiement du framework [5, 21].

6.5.2 Confiance proxy et authentification

Le bootstrap fait confiance à tous les proxies. Une exécution isolée du middleware, avec une adresse de connexion locale et une adresse différente dans l'en-tête transmis, montre que cette dernière devient l'adresse client vue par Laravel. Le middleware de limitation installé utilise cette adresse pour construire la clé des requêtes non authentifiées. Lorsque l'application est directement accessible, un client peut donc faire varier cette clé. Le risque est conditionnel au chemin réseau ; un proxy maîtrisé qui nettoie les en-têtes et interdit l'accès direct constitue une situation différente.

Les propositions sont de déclarer les proxies attendus et de vérifier la limitation avec la topologie réellement retenue. Par ailleurs, les routes de connexion et d'inscription ne régénèrent pas explicitement l'identifiant de session ; la déconnexion

oublie l'identité sans invalider entièrement la session. Il s'agit d'un risque de fixation si un attaquant parvient à faire utiliser une session qu'il connaît, et non d'une preuve de prise de compte réalisée. Les tests futurs devront vérifier le changement d'identifiant après authentification et l'inutilisabilité de la session déconnectée [14].

La synchronisation automatique des données de démonstration appelle une mesure complémentaire : supprimer son déclenchement pendant les lectures ordinaires et imposer une activation explicite du mode de démonstration. Les comptes de présentation et la commande qui réapplique un mot de passe par défaut à tous les utilisateurs doivent être exclus du fonctionnement normal. Aucun identifiant réel ni secret n'est reproduit dans ce mémoire. Le résultat recherché est l'absence de création ou de réinitialisation de compte provoquée indirectement par une consultation.

6.6 Cohérence métier et persistance : constats statiques

La route d'affichage d'une offre vérifie qu'un étudiant consulte une offre publiée. La route de candidature contrôle l'existence de l'offre et du profil, mais pas l'état de publication. Une demande directe peut donc suivre un chemin différent de celui suggéré par l'interface, notamment après fermeture d'une offre précédemment consultée. Le correctif proposé consiste à appliquer l'invariant au moment de l'écriture. Le test correspondant devra prouver qu'une candidature sur une offre fermée ou en brouillon ne crée aucun enregistrement.

La méthode SQL de mise à jour d'un profil modifie successivement le compte, le profil et le document sans transaction englobante. Le fichier est déplacé avant la sauvegarde métier. Un essai par injection d'échec devra vérifier la cohérence de l'état final et le nettoyage des fichiers devenus inutiles. Les transactions existantes pour certaines créations sont utiles, mais ne couvrent pas automatiquement toutes les opérations du magasin [4]. Il faut définir l'unité de cohérence de chaque cas d'utilisation, puis organiser les écritures et compensations en conséquence.

La candidature vérifie d'abord l'absence d'un enregistrement avant de l'insérer. La migration prévoit une contrainte d'unicité sur le couple offre-profil, ce qui protège la base contre un doublon persistant. Deux demandes simultanées peuvent néanmoins réussir leur vérification préalable et provoquer ensuite une exception sur l'une des insertions. Cette situation n'a pas été exécutée sur MySQL pendant l'audit. Le test proposé doit confirmer une réponse métier stable et idempotente, en s'appuyant sur la contrainte relationnelle plutôt qu'en supposant la séquentialité des utilisateurs [13].

Le repli JSON utilise des séquences de lecture, modification et réécriture sans verrouillage global de l'opération. Des mises à jour concurrentes peuvent donc s'écraser. Un JSON invalide déclenche aussi la reconstruction du jeu de démonstration. Ces

observations sont issues du code ; aucune perte de données existantes n'est affirmée. Pour la production, il convient de conserver un magasin autoritatif unique et de signaler son indisponibilité. Les essais de panne doivent vérifier qu'une erreur SQL ne conduit pas à annoncer une sauvegarde dans un univers de données différent.

Enfin, une consultation qui recalcule un score peut modifier le statut d'une candidature en présélection. Ce couplage doit être évalué comme une règle métier à part entière. La proposition est de séparer calcul et décision, de rendre explicite le déclenchement d'une présélection automatique et de journaliser sa cause. Un score conservé devrait référencer les versions du profil, de l'offre et du moteur ayant servi au calcul. Sans ces éléments, une variation ultérieure peut être difficile à expliquer au recruteur.

6.7 Protocole proposé d'évaluation de la pertinence

6.7.1 Constitution du corpus et annotations

Une évaluation scientifique nécessite un corpus distinct des données de démonstration. Les offres doivent couvrir plusieurs familles professionnelles et différents niveaux d'exigence. Les CV associés doivent comprendre des cas pertinents, partiellement pertinents et hors domaine. Les documents d'une même personne, les variantes quasi identiques et les doublons d'offres doivent rester dans la même partition, afin d'éviter une fuite entre le réglage et le test. Le référentiel, les pondérations et les seuils seront ajustés sur une partition de développement, puis figés avant l'évaluation finale.

La vérité de référence repose sur des annotations humaines explicites. Pour chaque couple offre–profil, les évaluateurs examinent les compétences nécessaires, les expériences professionnelles pertinentes, la formation lorsque l'offre la justifie et les éléments factuels du parcours. Le sexe, l'âge, l'origine et les autres caractéristiques sans rapport avec les exigences du poste ne doivent pas devenir des critères de pertinence. L'erreur « âge vers expérience » observée montre pourquoi la qualité de la transformation des données doit être évaluée avant leur utilisation dans un classement.

Une grille graduée peut distinguer absence d'adéquation, adéquation partielle et adéquation forte, à condition que chaque niveau soit défini avant l'annotation. Plusieurs évaluateurs annoteront indépendamment un sous-ensemble commun, puis documenteront leurs désaccords et leur procédure d'arbitrage. Aucun accord humain n'a été mesuré à ce stade. La littérature sur l'évaluation des outils de recrutement invite précisément à interroger la définition de la cible et les hypothèses incorporées aux mesures [18]. Un classement de proximité textuelle ne remplace pas une appréciation professionnelle contextualisée.

6.7.2 Mesures de classement

Pour une offre q , on note R_q l'ensemble des profils pertinents selon l'annotation, et $T_{q,k}$ les k premiers profils retournés. Les premières mesures proposées sont :

$$\text{Precision@ } k(q) = \frac{|T_{q,k} \cap R_q|}{k}, \quad (6.1)$$

$$\text{Recall@ } k(q) = \frac{|T_{q,k} \cap R_q|}{|R_q|}. \quad (6.2)$$

La précision mesure la part de profils pertinents parmi les premiers résultats consultés. Le rappel indique la proportion des profils pertinents retrouvée dans cette même fenêtre. Le protocole fixera des valeurs de k compatibles avec le travail des recruteurs et traitera explicitement les offres sans profil pertinent annoté. Les conventions concernant les listes plus courtes que k doivent également être définies avant le calcul. Les mesures seront d'abord calculées par offre, puis agrégées, pour éviter qu'une seule offre très représentée ne domine l'ensemble [9].

La NDCG permet de conserver les niveaux de pertinence et de valoriser leur position. Si $r_{q,i}$ est la pertinence graduée du profil placé en position i , on définit :

$$\text{DCG@ } k(q) = \sum_{i=1}^k \frac{2^{r_{q,i}} - 1}{\log_2(i + 1)}, \quad (6.3)$$

$$\text{NDCG@ } k(q) = \frac{\text{DCG@ } k(q)}{\text{IDCG@ } k(q)}. \quad (6.4)$$

Le dénominateur représente le meilleur ordre possible pour les mêmes annotations. Le cas d'un dénominateur nul sera signalé selon une convention documentée. Ces mesures comparent un ordre proposé à une référence indépendante ; elles sont donc différentes du score affiché par SmartRecruit. Aucun résultat de précision, rappel ou NDCG n'est fourni tant que ce corpus annoté n'a pas été constitué.

6.7.3 Comparaisons et analyse des erreurs

Le plan expérimental comparera une recherche lexicale simple, la couverture de compétences, le score hybride PHP et le score Python. Une étude d'ablation retirera successivement les synonymes, la proximité de catégories et le bonus d'expérience pour examiner leur contribution. Une évolution vers des représentations vectorielles apprises devra être soumise au même protocole, sans supposer qu'un modèle plus complexe améliore nécessairement la pertinence. Les résultats devront être accompagnés d'une analyse d'erreurs : synonymes absents, compétences ambiguës, extraction incorrecte et recommandations trop générales.

6.8 Plan de validation complémentaire et progression

La prochaine étape consiste à construire des tests fonctionnels avec une base jetable et des fichiers synthétiques. Ils couvriront inscription, connexion, propriétés des offres, visibilité des profils, candidatures et transitions de statut. Les scénarios d'échec introduiront une indisponibilité du service Python, une réponse JSON invalide, une contrainte SQL et une erreur de stockage. Chaque cas devra vérifier la réponse utilisateur et les effets réellement enregistrés. L'objectif est une cohérence observable, y compris lorsque plusieurs composants ne réussissent pas simultanément.

Les performances seront mesurées séparément après cette stabilisation. Les boucles de classement comparent plusieurs offres et plusieurs profils, avec des appels synchrones et des écritures éventuelles. Une campagne devra distinguer coût de calcul, coût réseau et coût SQL, puis observer plusieurs tailles de catalogue et niveaux de concurrence. Elle enregistrera médiane, percentiles élevés, erreurs et consommation de ressources. Les délais configurés dans le client ne sont pas des résultats de cette campagne ; aucun objectif chiffré de capacité n'est validé actuellement.

Table 6.3: Progression proposée à partir de l'évaluation du prototype

Priorité	Travaux proposés	Preuve attendue
Immédiate	Racine publique, isolation des données de démonstration, session et contrôle de candidature.	Tests de refus des chemins internes et des opérations interdites.
Fiabilité	Transactions, concurrence, diagnostics d'extraction et contrat partagé des moteurs.	Fixtures reproductibles, absence d'état partiel et résultats expliqués.
Pertinence	Corpus séparé, annotations professionnelles et comparaison des classements.	Mesures par offre et analyse documentée des erreurs.
Exploitation	Déploiement isolé, sauvegarde, observabilité et campagne de performance.	Procédure de reprise vérifiée et capacité mesurée.

L'évaluation confirme ainsi la présence d'un prototype cohérent dans son intention et suffisamment concret pour permettre des vérifications reproductibles. Elle révèle aussi des écarts précis entre continuité de démonstration, cohérence des scores et garanties attendues d'un service exploité. La progression proposée relie chaque limite à une correction et à une preuve future. Elle permet de présenter le travail avec ses résultats effectifs, tout en conservant une responsabilité humaine sur les décisions de recrutement et une traçabilité des évolutions du système.

Conclusion générale

SmartRecruit répond à un besoin de structuration de la rencontre entre étudiants et entreprises. Le prototype réunit les profils, les CV, les offres et les candidatures dans une application qui peut fournir un classement accompagné d'indicateurs sur les compétences. Le travail a permis de décrire une solution articulant développement web, stockage relationnel, extraction de texte et recherche d'information, au lieu de traiter le moteur de recommandation comme un composant isolé du parcours utilisateur.

L'architecture repose sur un backend Laravel, des vues Blade, une base MySQL et un service Python. La séparation du traitement de texte facilite l'évolution du moteur, tandis que le calcul PHP fournit un chemin de secours. La représentation par compétences, TF-IDF et catégories rend le score inspectable. Ces choix constituent une base pertinente pour un prototype académique, car ils permettent d'expliquer les étapes du rapprochement et de construire progressivement un protocole de comparaison.

L'analyse critique conduit toutefois à nuancer le bilan. La présence de deux implémentations ne garantit pas leur équivalence : les différences de dictionnaires et de pondérations modifient les résultats. Le parsing peut interpréter des informations hors contexte ou accepter du contenu non exploitable. Le fonctionnement du stockage JSON ne préserve pas, à lui seul, la cohérence avec MySQL. Enfin, la configuration de lancement et certaines opérations de démonstration doivent être isolées du fonctionnement destiné à accueillir de vrais utilisateurs.

Ces limites ne sont pas uniquement des détails d'implémentation. Elles touchent la signification du résultat et la confiance que l'on peut lui accorder. Un score de matching ne mesure ni la valeur d'une personne ni sa réussite future. Il indique une proximité calculée entre des informations disponibles, avec les conventions du moteur utilisé. La présélection doit donc rester interprétable, contestable et soumise à une appréciation humaine des éléments professionnels pertinents.

La priorité de poursuite concerne la sécurisation du déploiement, l'explicitation du mode de démonstration et l'intégrité des données. Une deuxième étape consiste à unifier le contrat de matching, à versionner les règles et à transformer les sondes en tests de régression. L'évaluation sur un corpus annoté pourra ensuite comparer la méthode actuelle à d'autres représentations, notamment des encodeurs de phrases adaptés au français. L'ajout d'OCR, de traitements asynchrones et d'une meilleure observabilité trouvera sa place lorsque les exigences et les critères de validation correspondants seront définis.

Le mémoire fournit ainsi un résultat technique et une méthode de progression. Il expose ce que le prototype réalise, les conditions dans lesquelles ses résultats ont été observés et les améliorations nécessaires pour aller plus loin. Cette articulation entre réalisation, preuve et réflexion critique constitue l'apport central du projet dans le cadre du Master Développement Logiciel.

Bibliographie

- [1] Robin Burke. Hybrid recommender systems: Survey and experiments. *User Modeling and User-Adapted Interaction*, 12:331–370, 2002. doi: 10.1023/A:1021240730564. URL <https://link.springer.com/article/10.1023/A:1021240730564>.
- [2] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of NAACL-HLT*, pages 4171–4186. Association for Computational Linguistics, 2019. doi: 10.18653/v1/N19-1423. URL <https://aclanthology.org/N19-1423/>.
- [3] Docker. Docker compose documentation. URL <https://docs.docker.com/compose/>. Consulté le 11 septembre 2026.
- [4] Laravel. Laravel 12.x documentation: Database – getting started, . URL <https://laravel.com/docs/12.x/database>. Consulté le 11 septembre 2026.
- [5] Laravel. Laravel 12.x documentation: Deployment, . URL <https://laravel.com/docs/12.x/deployment>. Consulté le 11 septembre 2026.
- [6] Laravel. Laravel 12.x documentation: Routing, . URL <https://laravel.com/docs/12.x/routing>. Consulté le 11 septembre 2026.
- [7] Laravel. Laravel 12.x documentation: Validation, . URL <https://laravel.com/docs/12.x/validation>. Consulté le 11 septembre 2026.
- [8] James Lewis and Martin Fowler. Microservices, 2014. URL <https://martinfowler.com/articles/microservices.html>. Consulté le 11 septembre 2026.
- [9] Christopher D. Manning, Prabhakar Raghavan, and Hinrich Schütze. *Introduction to Information Retrieval*. Cambridge University Press, 2008. URL <https://nlp.stanford.edu/IR-book/>. Édition en ligne des auteurs, consultée le 11 septembre 2026.
- [10] Louis Martin, Benjamin Muller, Pedro Javier Ortiz Suárez, Yoann Dupont, Laurent Romary, Éric Villemonte de la Clergerie, Djamé Seddah, and Benoît Sagot. CamemBERT: a tasty french language model. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 7203–7219, 2020. doi:

- 10.18653/v1/2020.acl-main.645. URL <https://aclanthology.org/2020.acl-main.645/>.
- [11] Tomas Mikolov, Kai Chen, Greg Corrado, and Jeffrey Dean. Efficient estimation of word representations in vector space. *arXiv preprint arXiv:1301.3781*, 2013. URL <https://arxiv.org/abs/1301.3781>.
- [12] Object Management Group. Unified modeling language, version 2.5.1, 2017. URL <https://www.omg.org/spec/UML/2.5.1>. Spécification, consultée le 11 septembre 2026.
- [13] Oracle. Mysql 8.0 reference manual: Foreign key constraints. URL <https://dev.mysql.com/doc/refman/8.0/en/create-table-foreign-keys.html>. Consulté le 11 septembre 2026.
- [14] OWASP Foundation. Session management cheat sheet, . URL https://cheatsheetseries.owasp.org/cheatsheets/Session_Management_Cheat_Sheet.html. Consulté le 11 septembre 2026.
- [15] OWASP Foundation. File upload cheat sheet, . URL https://cheatsheetseries.owasp.org/cheatsheets/File_Upload_Cheat_Sheet.html. Consulté le 11 septembre 2026.
- [16] Pallets Projects. Flask documentation, 3.1.x. URL <https://flask.palletsprojects.com/en/stable/>. Consulté le 11 septembre 2026.
- [17] pdfminer.six contributors. pdfminer.six documentation. URL <https://pdfminersix.readthedocs.io/en/latest/>. Consulté le 11 septembre 2026.
- [18] Manish Raghavan, Solon Barocas, Jon Kleinberg, and Karen Levy. Mitigating bias in algorithmic hiring: Evaluating claims and practices. In *Proceedings of the 2020 Conference on Fairness, Accountability, and Transparency*, pages 469–481, 2020. doi: 10.1145/3351095.3372828. URL <https://arxiv.org/abs/1906.09208>.
- [19] Nils Reimers and Iryna Gurevych. Sentence-BERT: Sentence embeddings using siamese BERT-networks. In *Proceedings of EMNLP-IJCNLP*, pages 3982–3992, 2019. doi: 10.18653/v1/D19-1410. URL <https://aclanthology.org/D19-1410/>.
- [20] Gerard Salton and Christopher Buckley. Term-weighting approaches in automatic text retrieval. *Information Processing & Management*, 24(5):513–523, 1988. doi: 10.1016/0306-4573(88)90021-0.
- [21] The PHP Group. Php manual: Built-in web server. URL <https://www.php.net/manual/en/features.commandline.webserver.php>. Consulté le 11 septembre 2026.